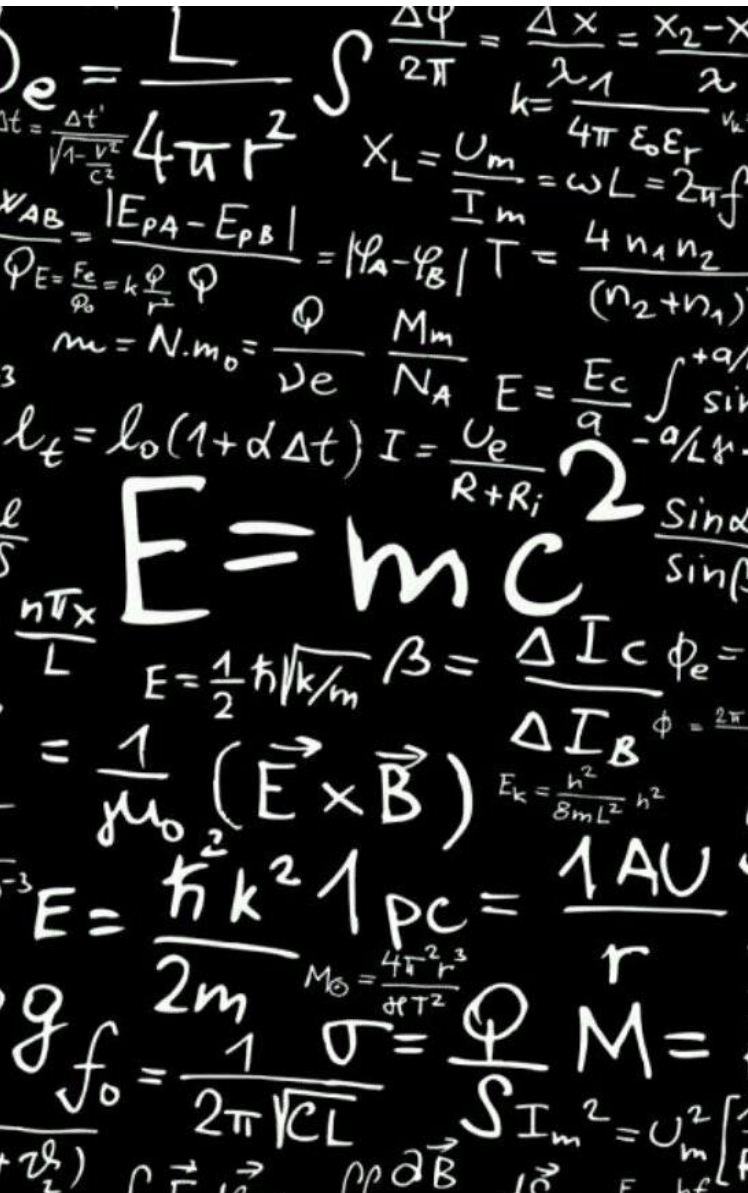




DATA PREPROCESSING And FEATURE ENGINEERING

Why do we care?



► Data Preprocessing

- Real world data almost always comes as a mess
- Some models have specific assumptions about inputs

► Feature Engineering

- Features in inputs sometimes are not enough or too many
- Insights from human knowledge or domain knowledge can improve accuracy

Data Preprocessing



- **Data Cleaning:** Real-world data is dirty — incomplete and inconsistent. We will need to perform cleaning operations such as filling in missing values, removing some outliers.
- **Data Integration:** We might need to pull data from several sources and make sure they are consistent.
- **Data Transformation:** We might need to transform data to fit machine learning model assumption such as scaling and standardizing.
- **Feature Selection:** Many features may be useless. Filtering irrelevant features out may improve the result.
- **Feature Extraction:** We need to transform data to help our machine learning model to learn from it, such as image processing.

Data Preprocessing



- **Data Transformation:** We might need to transform data to fit machine learning model assumption such as scaling and standardizing.

Problems with the dataset:

- Different scale
- Different distribution
- Skewed data
- Categorical data

Data Preprocessing



- ▶ **Data Cleaning:** Real-world data is dirty — incomplete and inconsistent. We will need to perform cleaning operations such as filling in missing values, removing some outliers.

Problems with the dataset:

- Missing Values
- Outliers

Data Preprocessing



- **Data Integration:** We might need to pull data from several sources and make sure they are consistent.

Problems with the dataset:

- Needed information are distributed across many tables

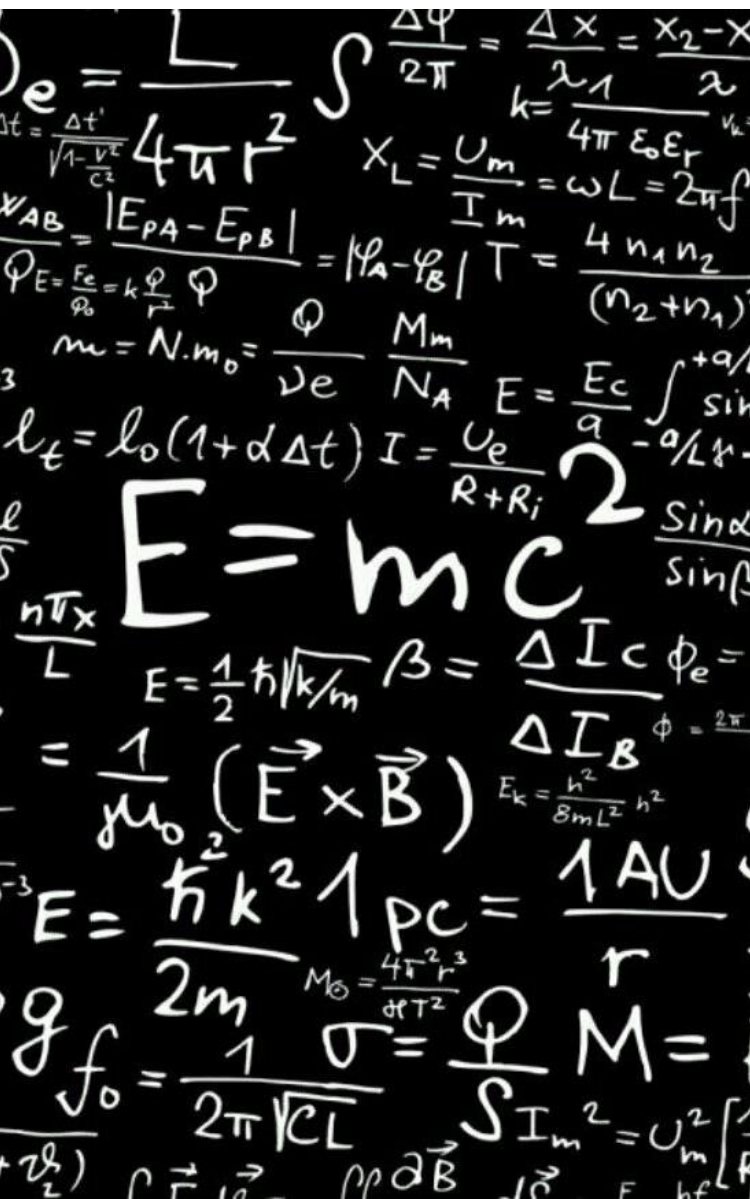
Data Preprocessing



- **Feature Selection:** Many features may be useless. Filtering irrelevant features out may improve the result.
- **Feature Extraction:** We need to transform data to help our machine learning model to learn from it, such as image processing.

Problems with the dataset:

- Irrelevant features
- Curse of dimensionality

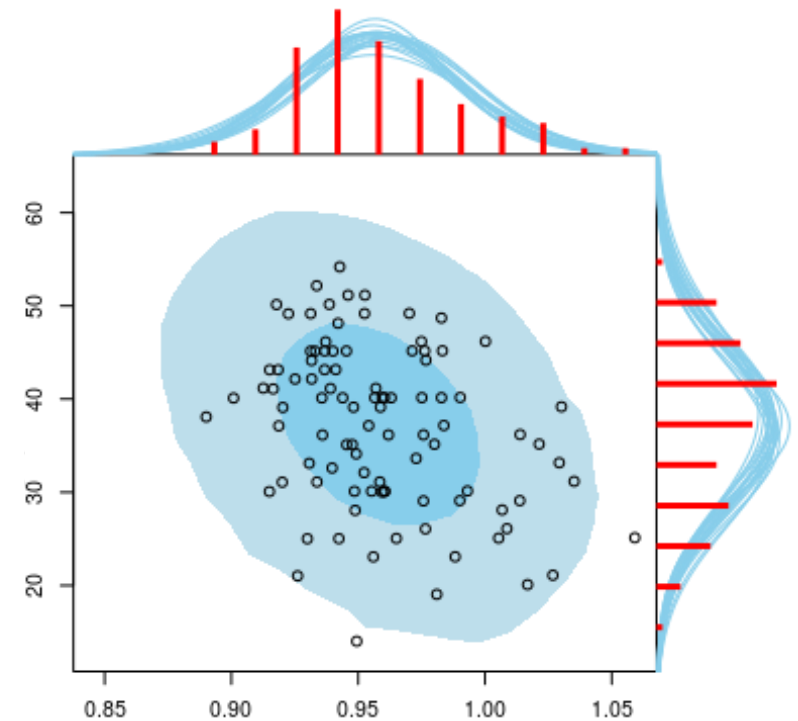


Model Assumption



It's important to be aware of model assumptions, when you use them.

- For example, regression has the following assumptions:
 - X and Y variables are numeric
 - X and Y variables have normal distributions (opt)
 - All variables must have the same variance (opt)



Feature Scaling



- Imagine if x_1 and x_2 are not similar in scale
- For example
 - x_1 = number of bedrooms (0-20)
 - x_2 = area of the house (24-3000 sqm)
- This means the scale of your theta will also be different.

Why Scaling



- Some algorithms like regression models and neural networks are very sensitive to scales.
- For example, if you want to compare the impact of number of bathrooms (scale 1-4) to the impact of areas (scale 24-250 sqm) on house prices, You need to rescale!

Feature Scaling



.....

normalization with max, min, mean

$$x_1 := \frac{x_1}{\max(x_1)}$$

$$x_1 := \frac{x_1 - \text{mean}(x_1)}{\max(x_1)}$$

$$x_1 := \frac{x_1 - \text{mean}(x_1)}{\max(x_1) - \min(x_1)}$$

**standardization
a.k.a. z-score**

$$x_1 := \frac{x_1 - \text{mean}(x_1)}{\text{std}(x_1)}$$

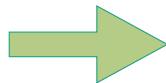
What's the range of these rescaled variables?

One-hot encoding



- When we have categorical data and our model requires numerical data, we need to transform text to numbers to feed into the model.
- A one-hot encoding operation transform categorical data into columns of 0 and 1.

Sample	Color			
1	Red	1	0	0
2	Green	0	1	0
3	Blue	0	0	1
...	...	...	...	...



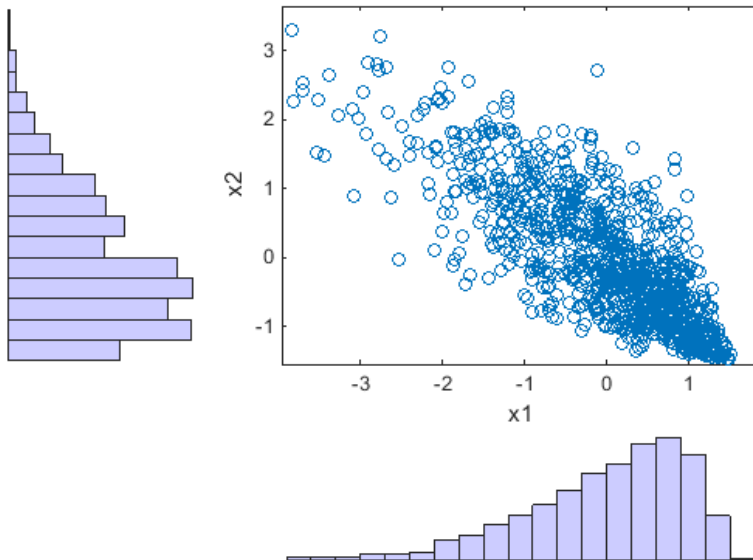
Sample	Is_Red	Is_Green	Is_Blue
1	1	0	0
2	0	1	0
3	0	0	1
...	...	...	...

Let's try this



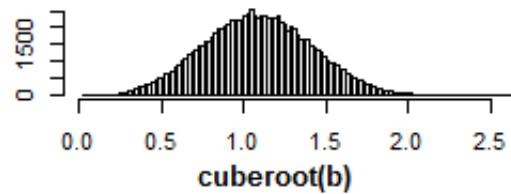
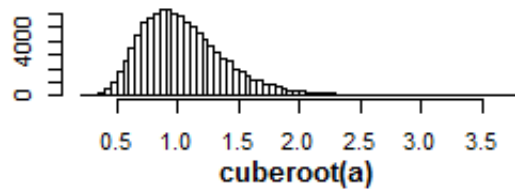
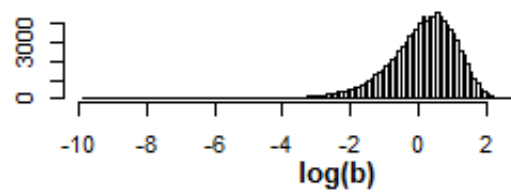
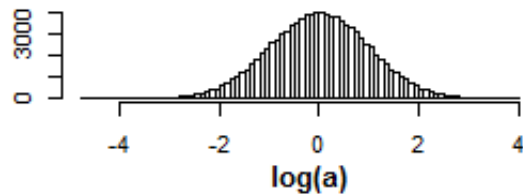
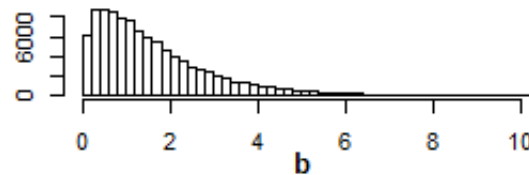
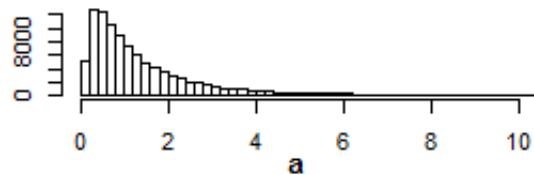
Sample	Grade
1	A
2	B
3	A
4	B+
5	F
6	D+
7	B
8	C
9	D+

Skewed Data



- Some algorithms like linear regression model assume normally distributed inputs.
- Many features do not behave that way!

Correcting Skewed data



► Applying non-linear function can fix skewed data

- Logarithm function
- Cube root
- Box-Cox

Let's Try this

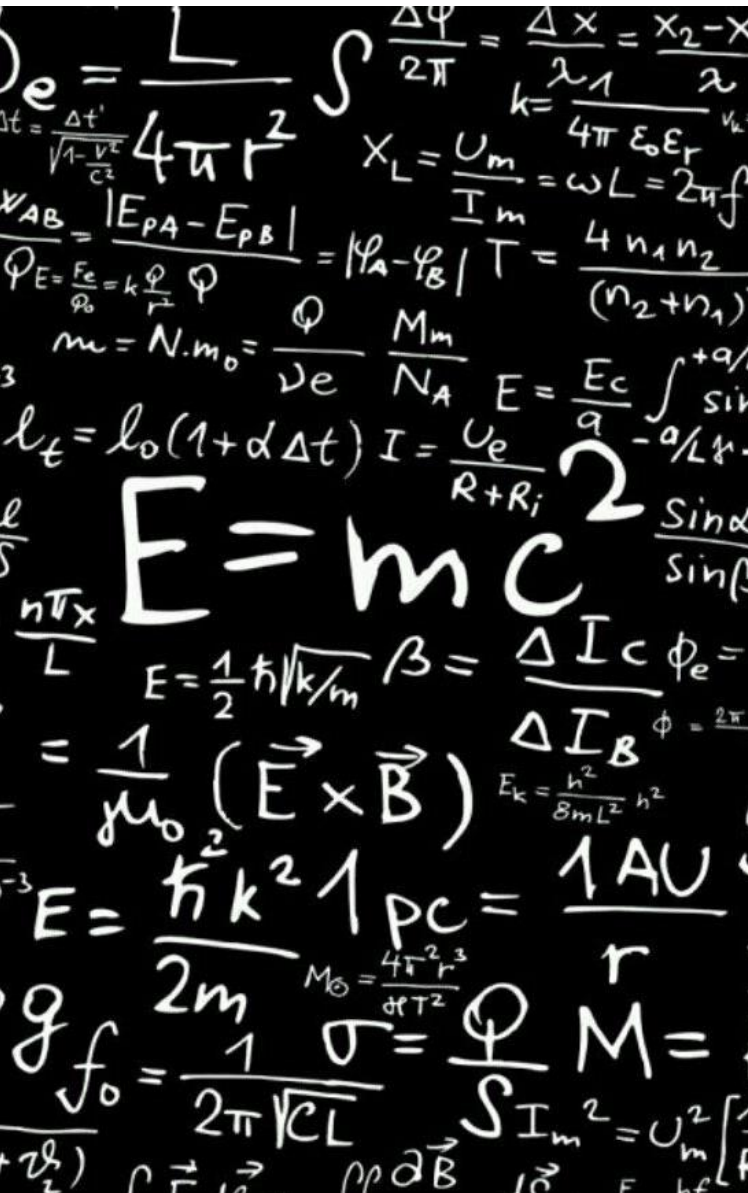


► Given the data:

1, 8, 8, 8, 27

What does the distribution look like?

How do we fix it?



DATA CLEANING

Missing Data



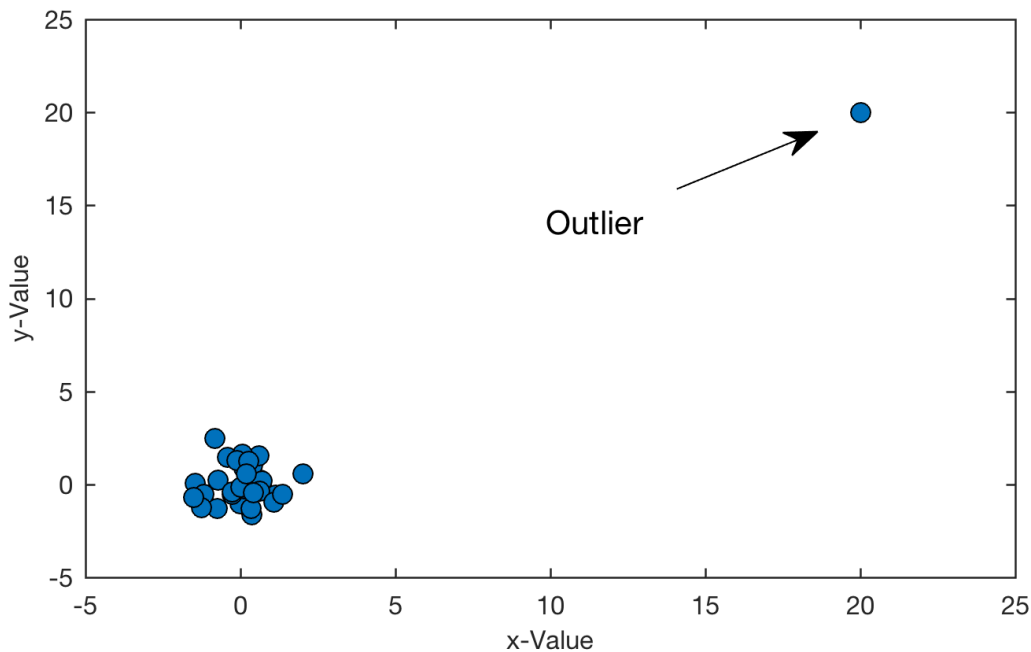
- If data are assumed to be missing at random, we may simply ignore the data
- In this case, you simply remove the missing data from the analysis (cut the whole row or column)
- Be aware that missing data might not be as random as you think

Missing Data



- Mean or mode substitution
 - Missing income? Fill it with average income?
 - Don't know if the patient is left-handed or right-handed, assume right-handed, because it's more common.
- Regression
- Nearest Neighbor

Outliers



- Outliers are those data points that are way different from the rest.
- Outliers can be misleading.
- For example, in this picture, without outliers you see no correlations in the data, but with outliers, correlations start to emerge, but those correlations are unreal!

What Cause Outliers?



- Measurement errors (mistyped data)
- Sensor errors
- Freaky circumstances (events that are unlikely, but happen, for example, a gigantic product sales due to liquidation).

Removing Outliers

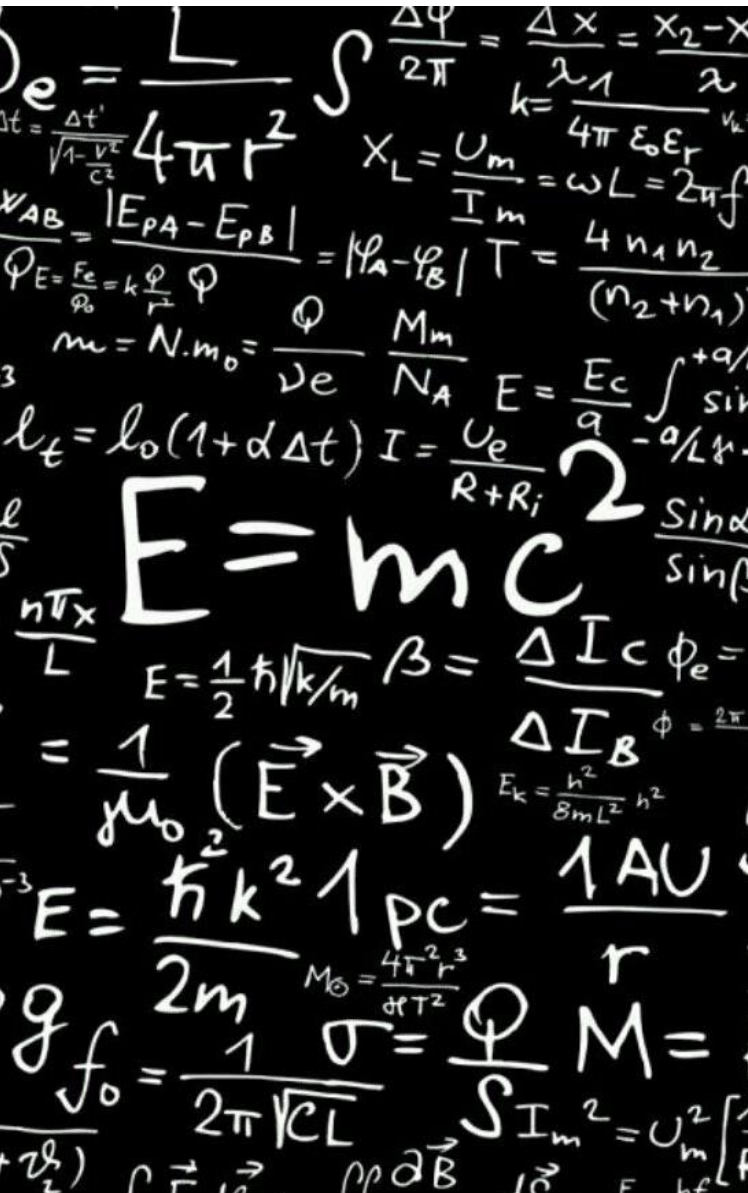


- Make a scatterplot or histogram plot of your data and look for extreme values.
- Assume that data has a Gaussian distribution and look for values more than 2-3 standard deviations from the mean.
- Filter out outliers candidate from training dataset and see if model's performance improve.

Outlier Removal Algorithm

A horizontal bar composed of five colored segments: blue, green, light green, orange, and red.

- Train model with all the data
- Look at data points with largest residual errors (say 10% of the data)
- Remove those data points from the dataset
- Retrain the model, do it over and over.



DIMENSIONALITY REDUCTION

Curse of Dimensionality



- This is one of the most important problems in machine learning.
 - Imagine you have a large amount of features, each can have infinite number of values.
 - You will need an enormous amount of training data is required to ensure that there are several samples with each combination of values.
 - If you have limited samples, which do not cover the whole space, your model loses predictive power.

Curse of Dimensionality



- This is one of the most important problems in machine learning.
 - Take linear regression for example
 - The more features you have, the more parameters you need to fit the model
 - If you have 2 features, your solution space has 2 dimensions (small possible values)
 - If you have 1000 features, your solution space has 1000 dimensions (huge amount of possible values).
 - Your algorithm can take a lot of time to find solution.

Dimensionality Reduction



Dimensionality Reduction: take very high-dimensional features and transform them into lower-dimension features without losing the quality of the data.

This is usually done by optimizing a cost function that quantifies goodness of components.

$$\begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ \dots \\ x_{100} \end{bmatrix} \longrightarrow \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix}$$

Dimensionality Reduction



Dimensionality Reduction

A process which takes very high-dimensional features and transform them into lower-dimension features without losing the quality of the data

Generally, this can be divided into

- Feature Extraction: Create few new features from old ones
- Feature Selection: Choose few important features

$$\begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ \dots \\ x_{100} \end{bmatrix} \longrightarrow \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix}$$

Feature transformation (Extraction)

- Principal Component Analysis (PCA)
- Linear Discriminant Analysis (LDA)
- Multidimensional Scaling (MDS)
- Random Projections (R)
- t-distributed stochastic neighbor embedding (t-SNE)
- Uniform manifold approximation and projection (UMAP)

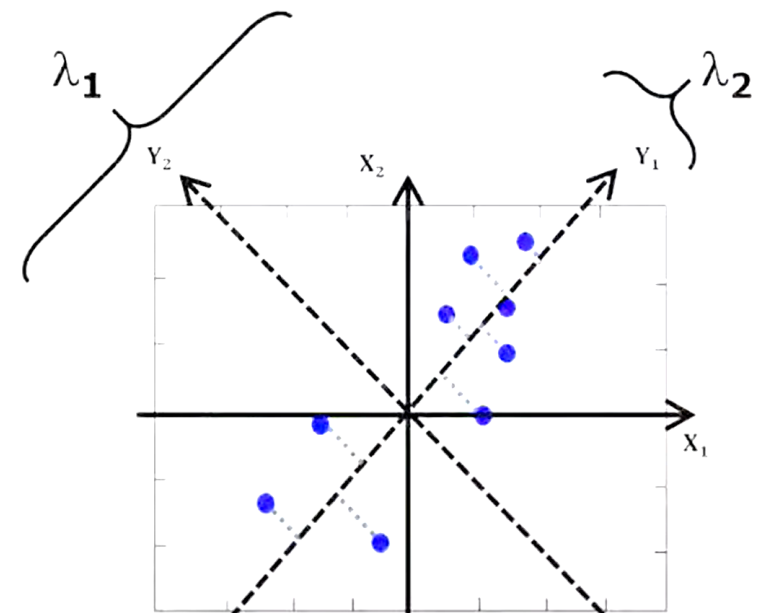
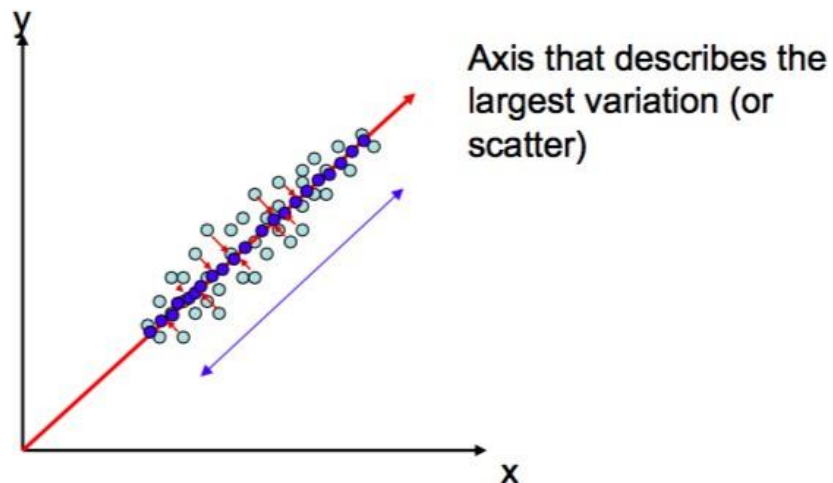
Why Principal Component Analysis



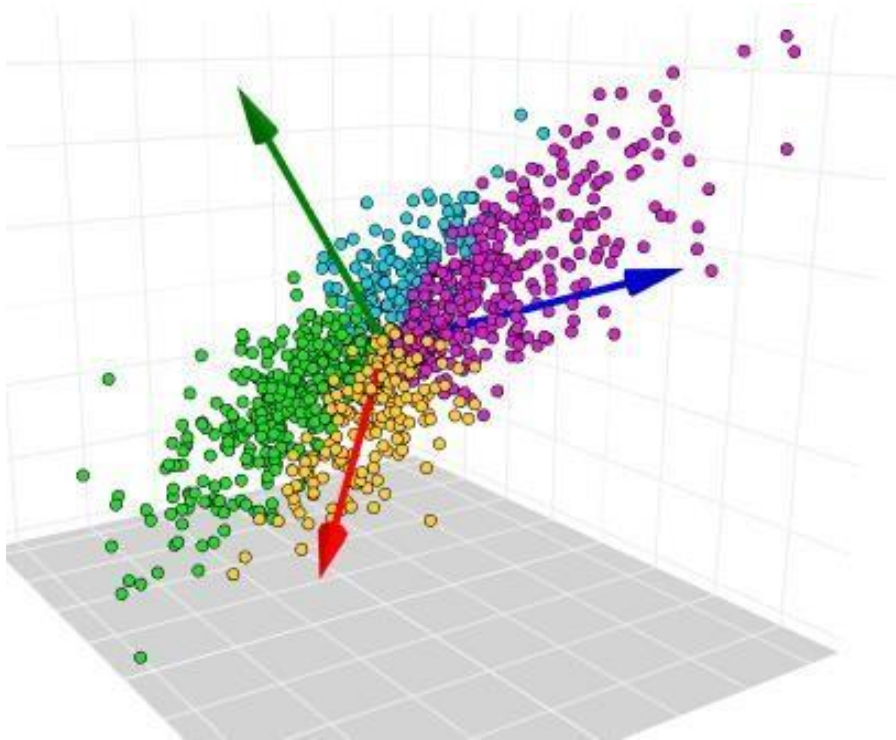
- Visualization
- More efficient use of resources
- Noise and outlier removal
- Faster processing by machine learning algorithm

What is PCA?

- We want to reduce the dimensionality but keep useful information
 - What is useful information? Variation
- We want to find a projection (a transformation) that
- describe maximum variation

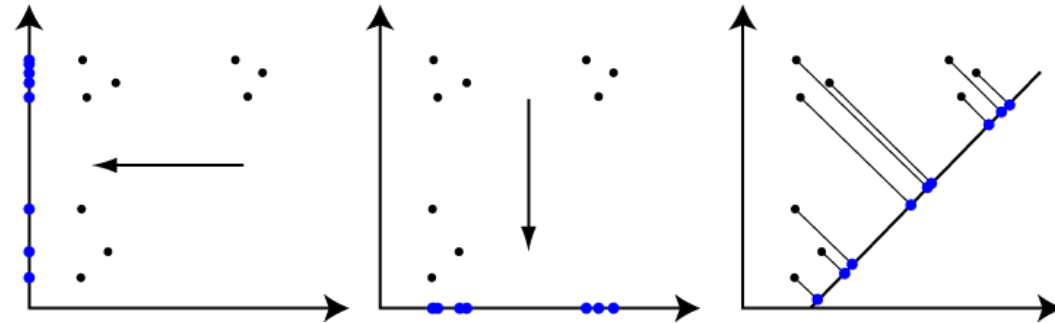


Variance is information



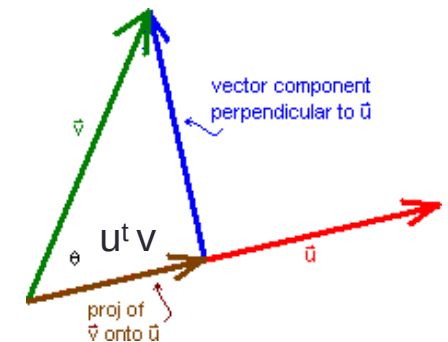
<https://towardsdatascience.com/principal-component-analysis-pca-explained-visually-with-zero-math-1cbf392b9e7d>

Projection



<https://shapeofdata.wordpress.com/2013/04/16/visualization-and-projection/>

- High variances = more power to distinguish groups of data points



Formulation

- Maximize the variance after projection ie
 - $\operatorname{argmax} \operatorname{Var}(\mathbf{w}^t \mathbf{x})$
- Subject to $\mathbf{w}$ is a unit vector

- $\Sigma \mathbf{w} = \lambda \mathbf{w}$ <- eigenvector

Covariance matrix

- Given a set of RVs, $X_1 X_2 \dots X_n$
- The covariance matrix is a matrix which has the covariance of the i and j RV in position (i,j)

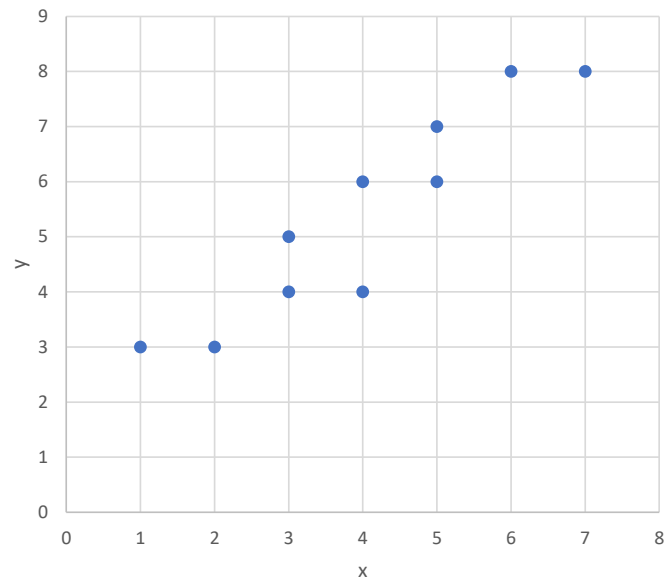
$$\Sigma = \begin{bmatrix} E[(X_1 - \mu_1)(X_1 - \mu_1)] & E[(X_1 - \mu_1)(X_2 - \mu_2)] & \cdots & E[(X_1 - \mu_1)(X_n - \mu_n)] \\ E[(X_2 - \mu_2)(X_1 - \mu_1)] & E[(X_2 - \mu_2)(X_2 - \mu_2)] & \cdots & E[(X_2 - \mu_2)(X_n - \mu_n)] \\ \vdots & \vdots & \ddots & \vdots \\ E[(X_n - \mu_n)(X_1 - \mu_1)] & E[(X_n - \mu_n)(X_2 - \mu_2)] & \cdots & E[(X_n - \mu_n)(X_n - \mu_n)] \end{bmatrix}.$$

PCA Summary



PCA Example

x	y
1	3
2	3
3	4
3	5
4	4
4	6
5	6
5	7
6	8
7	8
$\bar{x} = 4$	$\bar{y} = 5.4$



PCA Example

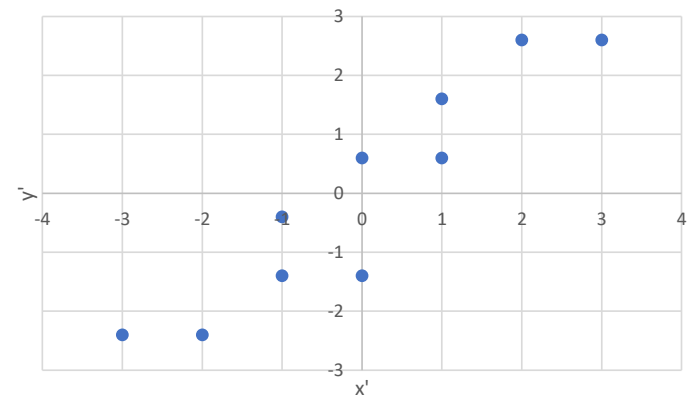
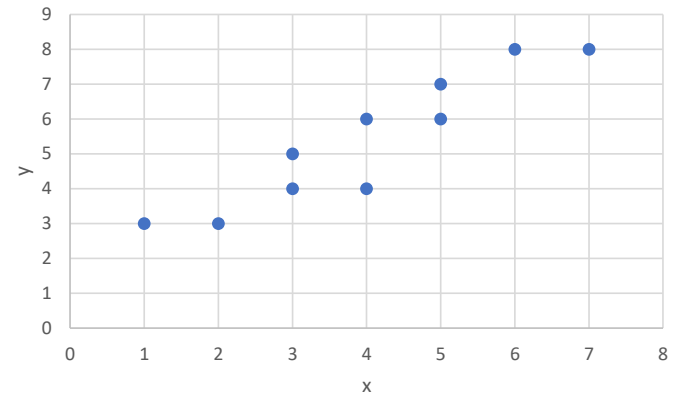
- Data Normalization

x	y
1	3
2	3
3	4
3	5
4	4
4	6
5	6
5	7
6	8
7	8
$\bar{x} = 4$	$\bar{y} = 5.4$

Data
Normalization



x'	y'
-3	-2.4
-2	-2.4
-1	-1.4
-1	-0.4
0	-1.4
0	0.6
1	0.6
1	1.6
2	2.6
3	2.6

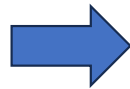


PCA Example

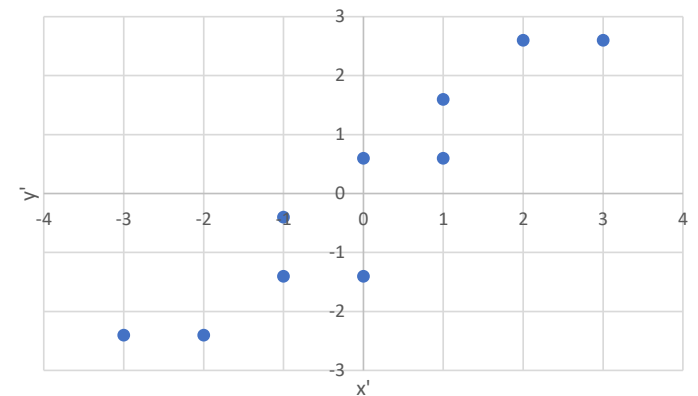
$$\text{Covariance Matrix } (C) = \frac{1}{M-1} AA^T$$

- Compute Covariance Matrix

A^T	
x'	y'
-3	-2.4
-2	-2.4
-1	-1.4
-1	-0.4
0	-1.4
0	0.6
1	0.6
1	1.6
2	2.6
3	2.6



C	
3.33	3.22
3.22	3.60



PCA Example

λ เป็นค่าเฉพาะของ A ก็ต่อเมื่อ $(A - \lambda I_n)\vec{x} = \vec{0}_n$ มีผลเฉลยไม่ซ้ำ
ก็ต่อเมื่อ $(A - \lambda I_n)$ ไม่มีเมทริกซ์ผกผัน
ก็ต่อเมื่อ $\det(A - \lambda I_n) = 0$

- Compute Eigenvalues and Eigenvectors

- $Cy_j = \lambda y_j \rightarrow (C - \lambda_j I)y_j = 0$

- $\begin{bmatrix} 3.33 - \lambda & 3.22 \\ 3.22 & 3.6 - \lambda \end{bmatrix} y_j = 0$

- $\det\left(\begin{bmatrix} 3.33 - \lambda & 3.22 \\ 3.22 & 3.6 - \lambda \end{bmatrix}\right) = 0$

- $(3.33 - \lambda)(3.6 - \lambda) - (3.22)(3.22) = 0 \rightarrow \lambda_1 = 0.242, \lambda_2 = 6.692$

C

3.33	3.22
3.22	3.60

PCA Example

λ เป็นค่าเฉพาะของ A ก็ต่อเมื่อ $(A - \lambda I_n)\vec{x} = \vec{0}_n$ มีผลเฉลยไม่ชัด
 ก็ต่อเมื่อ $(A - \lambda I_n)$ ไม่มีเมทริกซ์ผกผัน
 ก็ต่อเมื่อ $\det(A - \lambda I_n) = 0$

- Compute Eigenvalues and Eigenvectors

- *Case* : $\lambda_1 = 0.242$

- $$\begin{bmatrix} 3.33 - 0.242 & 3.22 \\ 3.22 & 3.6 - 0.242 \end{bmatrix} y_j = \begin{bmatrix} 3.33 - 0.242 & 3.22 \\ 3.22 & 3.6 - 0.242 \end{bmatrix} \begin{bmatrix} a \\ b \end{bmatrix} = 0$$

- $$\begin{bmatrix} a \\ b \end{bmatrix} = \begin{bmatrix} -0.722 \\ 0.692 \end{bmatrix}$$

Eigenvectors

-0.722	-0.692
0.692	-0.722

y_1

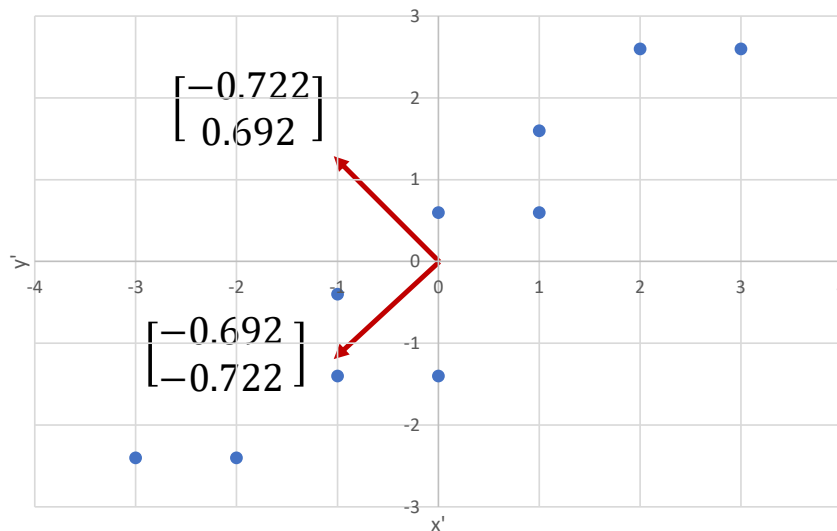
y_2

Eigenvalues

$$\lambda_1 = 0.242, \lambda_2 = 6.692$$

PCA Example

- Rotate axis and eliminate dimension

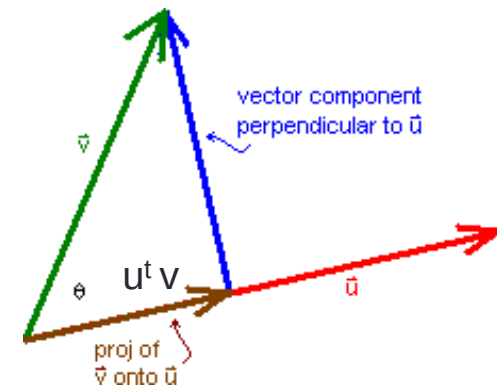


Eigenvalues

$$\lambda_1 = 0.242, \lambda_2 = 6.692$$

Eigenvectors

-0.692	-0.722
-0.722	0.692
y_2	y_1



PCA Example

- Rotate axis and eliminate dimension

A^T			$(A')^T$	
x'	y'		x'	y'
-3	-2.4	$A' = y^T A$ 	3.809	0.505
-2	-2.4		3.117	-0.217
-1	-1.4		1.703	-0.247
-1	-0.4		0.981	0.445
0	-1.4		1.011	-0.969
0	0.6		-0.433	0.415
1	0.6		-1.125	-0.307
1	1.6		-1.847	0.385
2	2.6		-3.261	0.355
3	2.6		-3.953	-0.367

PCA Example

- Rotate axis and eliminate dimension (Inverse)

x'
3.809
3.117
1.703
0.981
1.011
-0.433
-1.125
-1.847
-3.261
-3.953

M x 1

×

Eigenvectors (y₂)

-0.692	-0.722
--------	--------

1 x 2

=

x'	y'
-2.63583	-2.7501
-2.15696	-2.25047
-1.17848	-1.22957
-0.67885	-0.70828
-0.69961	-0.72994
0.299636	0.312626
0.7785	0.81225
1.278124	1.333534
2.256612	2.354442
2.735476	2.854066

M x 2

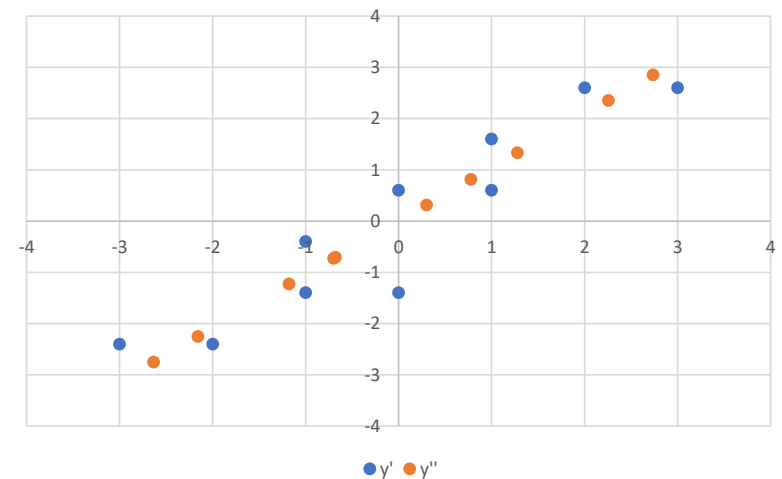
≈

x'	y'
-3	-2.4
-2	-2.4
-1	-1.4
-1	-0.4
0	-1.4
0	0.6
1	0.6
1	1.6
2	2.6
3	2.6

PCA Example

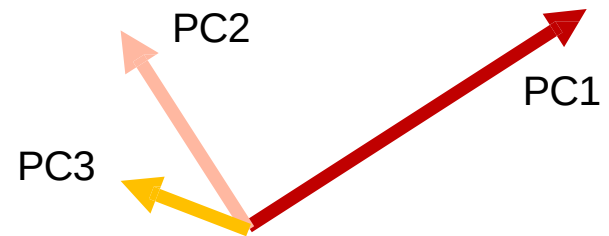
- Rotate axis and eliminate dimension (Inverse)

x'	y'		x'		x'	y'
-3	-2.4	PCA 	3.809	Inverse 	-2.63583	-2.7501
-2	-2.4		3.117		-2.15696	-2.25047
-1	-1.4		1.703		-1.17848	-1.22957
-1	-0.4		0.981		-0.67885	-0.70828
0	-1.4		1.011		-0.69961	-0.72994
0	0.6		-0.433		0.299636	0.312626
1	0.6		-1.125		0.7785	0.81225
1	1.6		-1.847		1.278124	1.333534
2	2.6		-3.261		2.256612	2.354442
3	2.6		-3.953		2.735476	2.854066

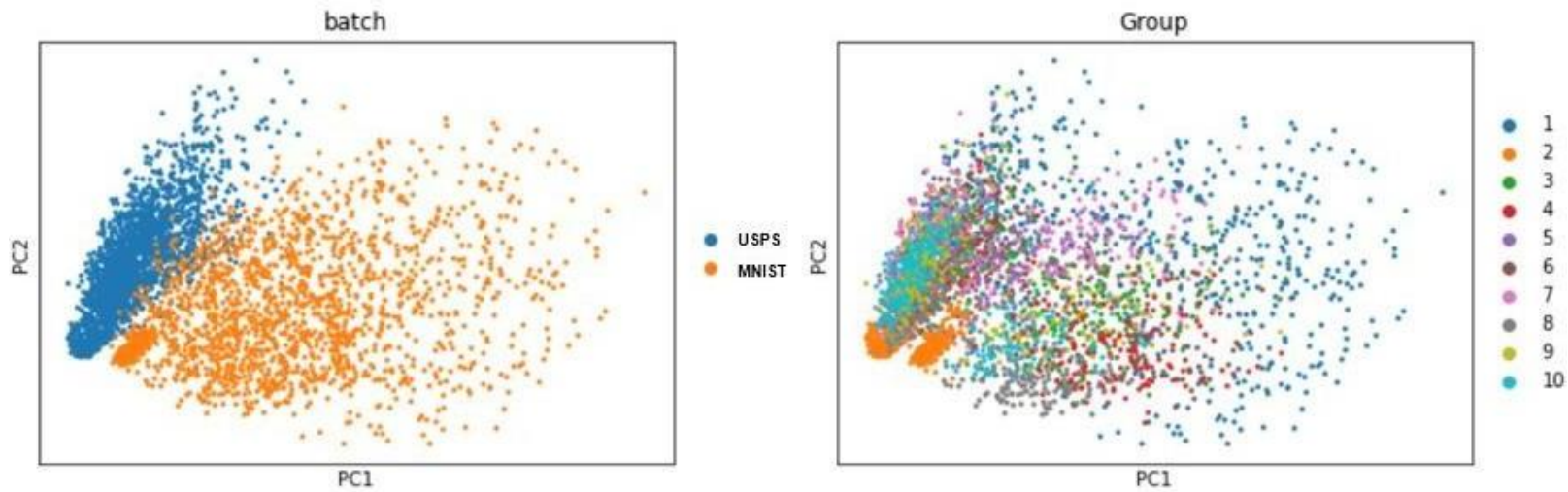


Interpretations of PCA algorithm

- Orthogonal linear transformation
 - New axes are **rotations** of the original axes
 - $PC1 = w_1x_1 + w_2x_2 + \dots + w_nx_n$
 - $PC2 = v_1x_1 + v_2x_2 + \dots + v_nx_n$
 - w_i 's and v_i 's are called **loadings**



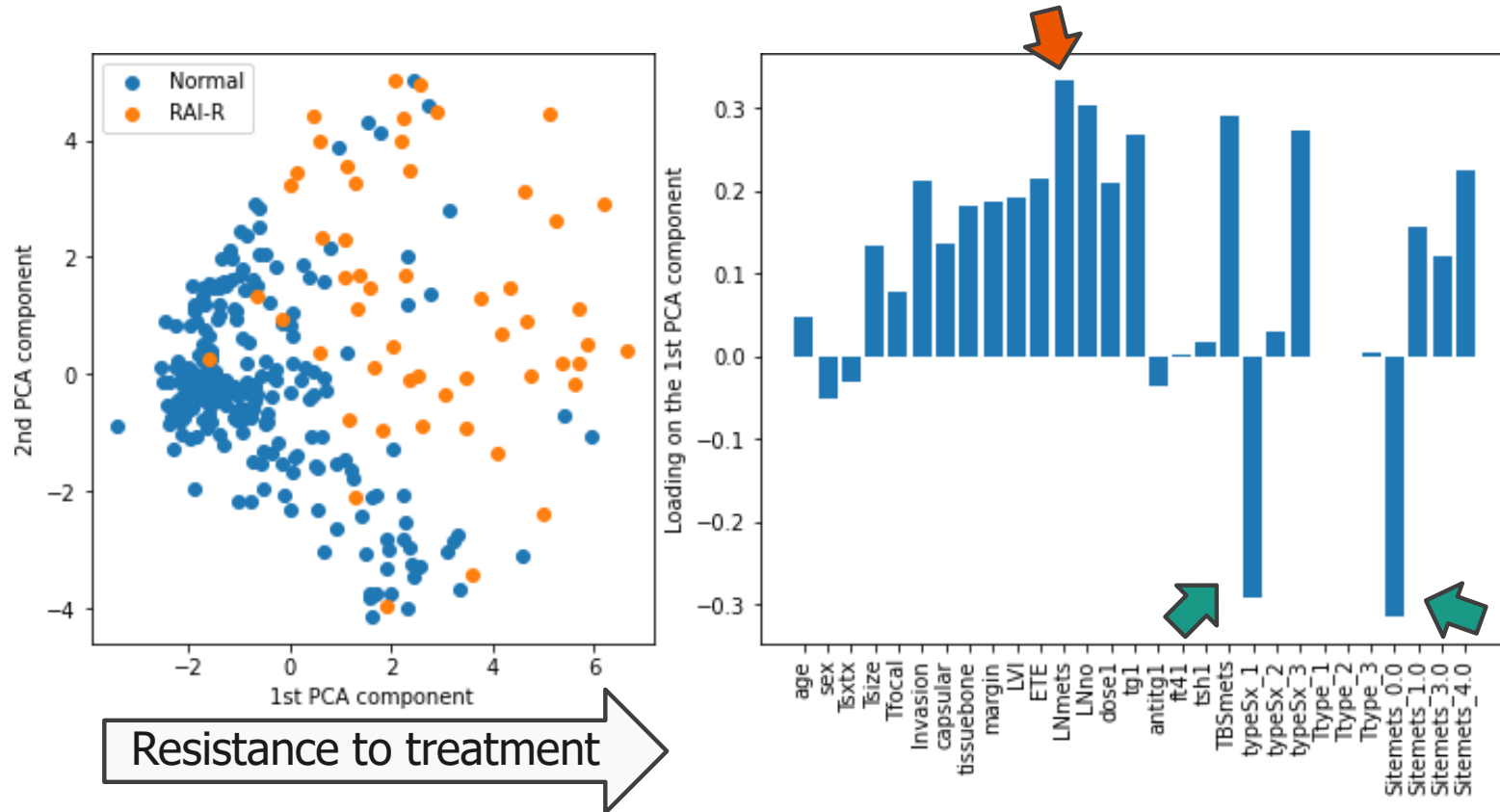
Interpretation of PCA result



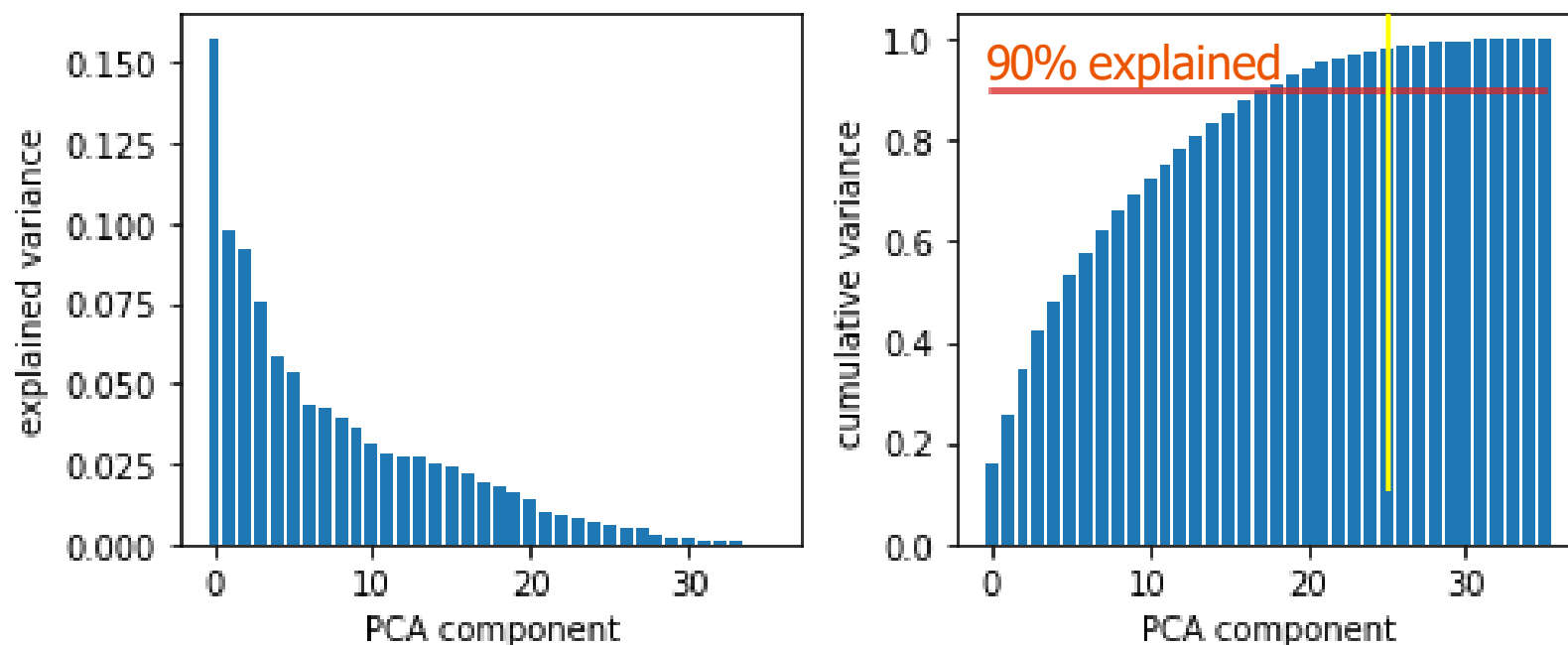
- PC1 captures the variance between data sources
- PC2 somewhat captures the variance between digit identity

<https://twitter.com/lkmklsmn/status/1436357177887895555>

Interpreting loadings on individual PC



PCA for dimensionality reduction



- By default, PCA retains the number of dimensions
- We can **select only the first k PC** for downstream analyses

PCA in Python

sklearn.decomposition.PCA

```
class sklearn.decomposition.PCA(n_components=None, *, copy=True, whiten=False, svd_solver='auto', tol=0.0,
iterated_power='auto', n_oversamples=10, power_iteration_normalizer='auto', random_state=None)
```

- **n_components** = number of directions/axes to produce
 - default = min(sample, feature)
- Input data will be **automatically centered** (set mean = 0 for each feature) **but not scaled** (set variance = 1 for each feature)
- **random_state** controls the behavior of algorithm that have some randomness
 - Always set to a fixed value for reproducibility

PCA usage

```
from sklearn.decomposition import PCA

pca = PCA(random_state = 25)
pca.fit(input_data)

## a fitted PCA can be used to transform any new
data input_embed = pca.transform(input_data)
output_embed = pca.transform(output_data)

## you can fit & transform directly
new_embed = pca.fit_transform(new_data)
```

PCA output

number of output directions/axes

`pca.n_components_`

array of variance explained by each axis (from high to low, sum to one)

`pca.explained_variance_ratio_`

2D array of the loading (w_i 's) of each feature on each
direction/axis ## dimension = # direction x # feature

`pca.components_`

the loading (w_i 's) of each feature on the 3rd

direction/axis `pca.components_[2]`

Big Data



- Most problems you will face in the real world is gigantic.
 - Millions of rows
 - Hundreds or thousands of features
 - Your algorithm will take forever to run
- What can we do about it?
 - We might be able to look through all the features and manually select them.
 - But that would waste so much time and resources
 - So maybe do automated feature selection?

Feature Selection



A process of selecting relevant, most important features to be used in machine learning model construction

$$\begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ \dots \\ x_{100} \end{bmatrix} \longrightarrow \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix}$$

Important Features



- The algorithm predicts whether the email is spam or not. Which feature is most useful for the prediction?
 - Feature 1: whether the email contains the word 'viagra'
 - Feature 2: whether the email is sent from a Nigerian Prince
 - Feature 3: whether the email is sent from one person to a massive amount of people
- For all 1000 features you have calculated, maybe only a few features are important.
- Feature selection algorithm gives you insight and interpretability of your model.

Dimensionality Reduction

Feature Selection

Filter Methods

- Correlation with target
- Variance threshold
- ...

Embedded Methods

- L1 (LASSO) regularization
- Decision tree
- ...

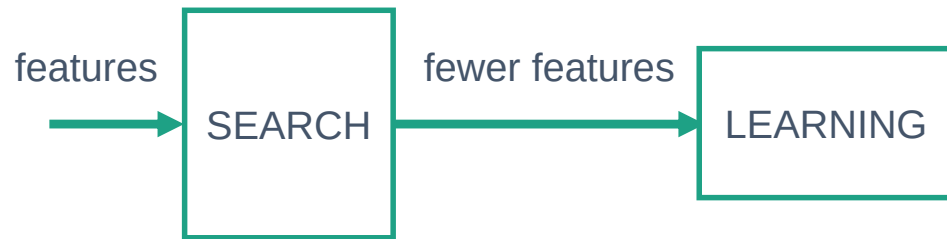
Wrapper Methods

- Recursive Feature Elimination (RFE)
- Sequential Feature Selection (SFS)
- Permutation importance
- ...

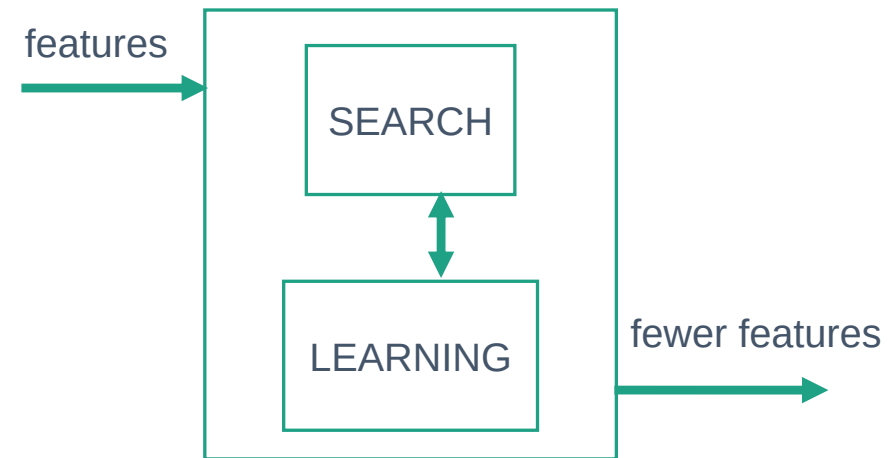
Feature Selection



Filtering Methods

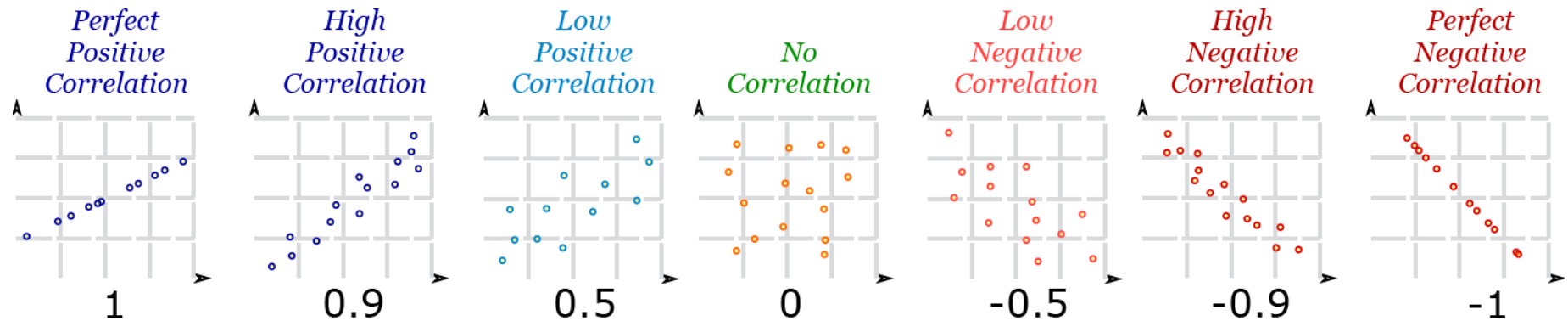


Wrapping Methods



Good Features?

5. Pearson (r_{xy})	Conti	Conti
6. Spearman's Rank (R_s)	Ordinal / Conti	Ordinal / Conti
7. Eta (η)	Nominal	Conti
8. Cramer's V	Nominal	Nominal/Ordinal



- You need to find features that have high correlation to your target.
 - Don't care whether it's a positive or negative correlation
 - The larger the number the better

Correlation

- Unit measure of change between two variables change with respect to each other.
- A normalized form of covariance.

$$\text{corr}(X, Y) = \frac{\text{cov}(X, Y)}{S_X S_Y}$$

- The value of correlation lies between -1 and $+1$.
 - If the correlation coefficient is one, it means that if one variable moves a given amount, the second moves proportionally in the same direction.
 - If correlation coefficient is zero, no relationship exists between the variables.
 - If correlation coefficient is -1, it means that one variable increases, the other variable decreases proportionally.

Good Features



- Correlation is not the only measure that tells you 'how much x is related to y' there are other measures we use.
- Such as:
 - ANOVA (Analysis of Variance)
 - Mutual Information
 - Chi2

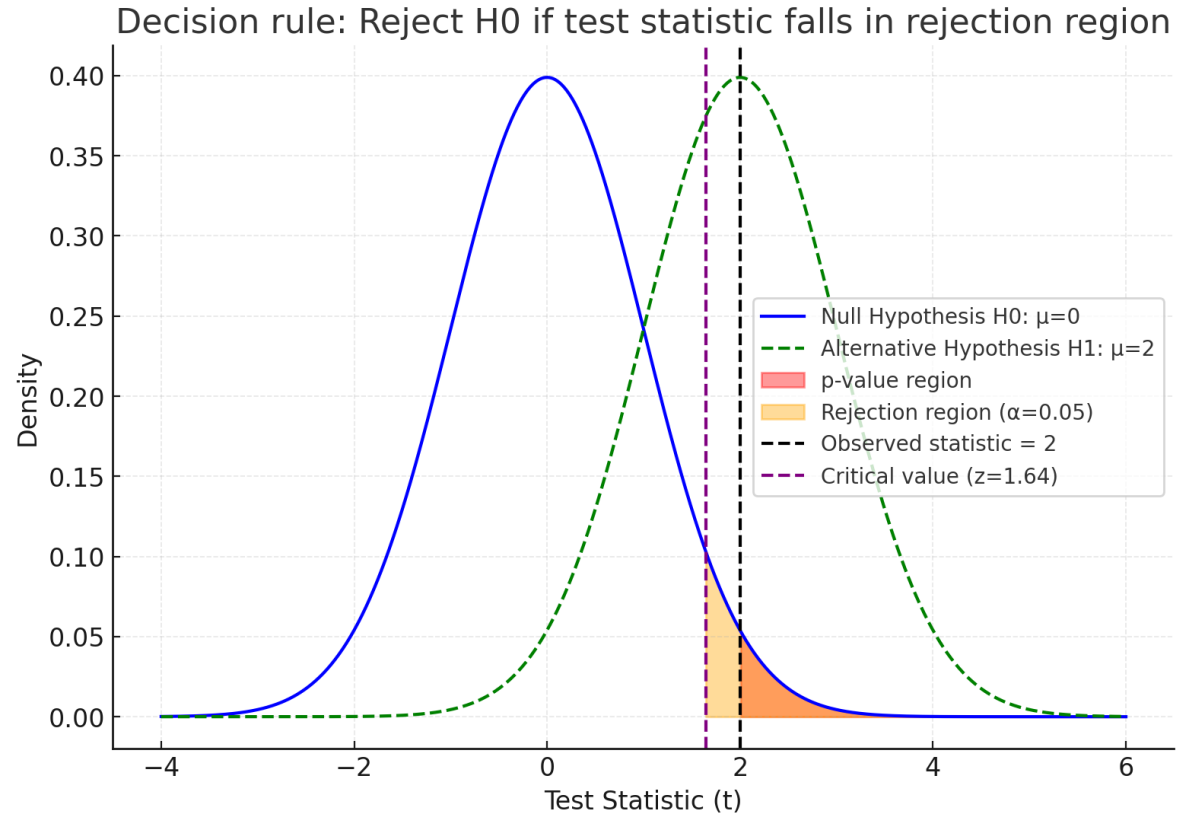
ANOVA (Analysis of Variance)

- ANOVA is used to test whether there are significant differences between the means of multiple groups by comparing the variance between groups with the variance within groups.
- **Hypotheses:**
 - **H_0 (Null Hypothesis):** The means of all groups are equal.
 - **H_1 (Alternative Hypothesis):** At least one pair of group means are different.
- **F-statistic:** Calculated by dividing the variance between groups by the variance within groups. The higher the F value, the more likely the means of the groups differ.
- **p-value:** Used to assess the likelihood that the observed result occurred by chance. If the p-value is less than 0.05, it indicates a statistically significant difference between groups.
- **Usage:** ANOVA is commonly used when comparing the means of more than two groups, such as comparing test scores of students from three different schools.

P-value

- ในการทดสอบสมมติฐาน ค่าความเบี่ยงเบนของข้อมูลจาก **null hypothesis** จะถูกสรุปด้วยค่า **p-value** ซึ่งเป็นความน่าจะเป็นที่จะสังเกตเห็นบางสิ่ง (เช่น ข้อมูล, ค่าเฉลี่ย หรือค่าสถิติทดสอบ อื่นๆ) ภายใต้ **null hypothesis**

$$p = Pr(D|H_0)$$



Kruskal Wallis test

ใช้ทดสอบความแตกต่างของ median >2 ประชากรที่เป็นอิสระต่อกัน
-ใช้แทน Oneway Anova เมื่อข้อมูลแต่ละประชากรไม่แจกแจงปกติ หรือ
ข้อมูลเป็นระดับ Ordinal

Key assumptions of ANOVA

- Assumption of Normality:
 - The data within each group should be normally distributed, or at least not significantly deviate from normality, especially when the sample size in each group is small.
- Assumption of Homogeneity of Variance (Homoscedasticity):
 - The variances within each group should be approximately equal.
- Assumption of Independence:
 - The observations within each group should be independent of each other, meaning that the value of one group should not influence or correlate with the values in another group.

Analysis of Variance (ANOVA)



- Classification problem: Y can only be class 0 or class 1. Find variance of X within class and between classes.

$$\text{F-Value} = \frac{\text{Variance between classes}}{\text{Variance within class}}$$

V between class is **high**
V within class is **low**
F-Value is **high**
Feature X is **important**

V between class is **low**
V within class is **high**
F-Value is **low**
Feature X is **not important**

$$SS_{\text{between}} = n(\bar{X}_A - \bar{X})^2 + n(\bar{X}_B - \bar{X})^2 + n(\bar{X}_C - \bar{X})^2$$

$$SS_{\text{within}} = \sum_{i=1}^k \sum_{j=1}^n (x_{ij} - \bar{X}_i)^2$$

Analysis of Variance (ANOVA)



Weight	Class
50	Adult
80	Adult
12	Children
30	Children
...	...

V between class is ...
V within class is ...
F-Value is ...
Feature is ...

Weight	Class
68	Thailand
75	China
80	Thailand
82	China
...	...

V between class is ...
V within class is ...
F-Value is ...
Feature is ...

Weight	Class
65	Human
1000	Animal
0.1	Animal
60	Animal
30	Human

V between class is ...
V within class is ...
F-Value is ...
Feature is ...

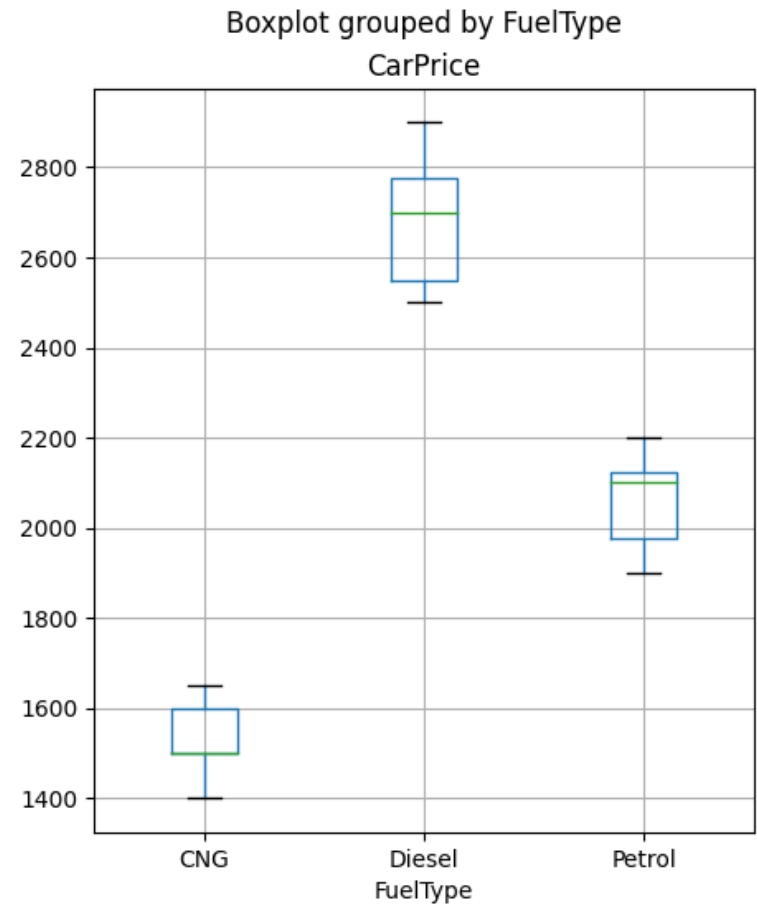
```

# Generating sample data
import pandas as pd
ColumnNames=['FuelType','CarPrice']
DataValues= [[ 'Petrol', 2000],
[ 'Petrol', 2100],
[ 'Petrol', 1900],
[ 'Petrol', 2150],
[ 'Petrol', 2100],
[ 'Petrol', 2200],
[ 'Petrol', 1950],
[ 'Diesel', 2500],
[ 'Diesel', 2700],
[ 'Diesel', 2900],
[ 'Diesel', 2850],
[ 'Diesel', 2600],
[ 'Diesel', 2500],
[ 'Diesel', 2700],
[ 'CNG', 1500],
[ 'CNG', 1400],
[ 'CNG', 1600],
[ 'CNG', 1650],
[ 'CNG', 1600],
[ 'CNG', 1500],
[ 'CNG', 1500]]

#Create the Data Frame
CarData=pd.DataFrame(data=DataValues,columns=ColumnNames)
print(CarData)

# Generating boxplot for CarPrice Vs FuelType
CarData.boxplot(column='CarPrice', by='FuelType', figsize=(5,6))

```



```

# f_oneway() function takes the group data as input and
# returns F-statistic and P-value
from scipy.stats import f_oneway

# Running the one-way anova test between CarPrice and
FuelTypes
# Assumption(H0) is that FuelType and CarPrices are NOT
correlated

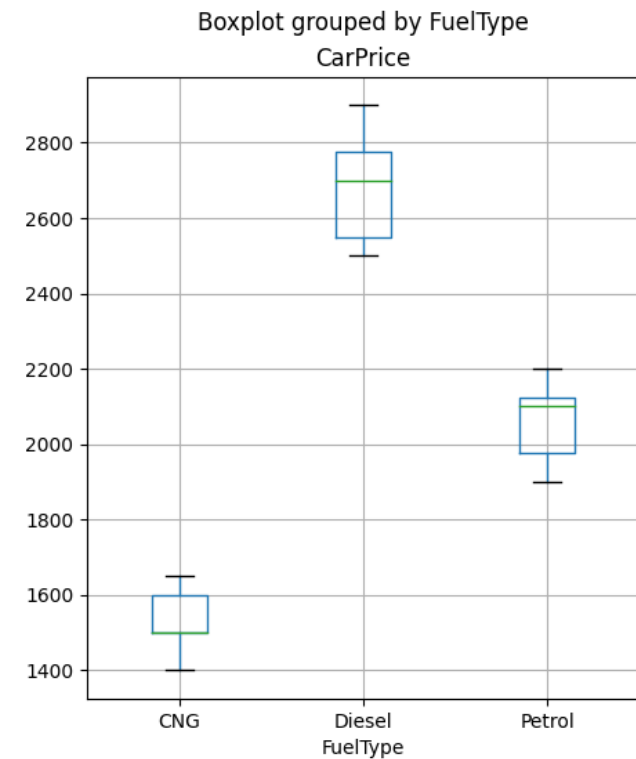
# Finds out the Prices data for each FuelType as a list
CategoryGroupLists=CarData.groupby('FuelType')['CarPrice'].apply(list)

# Performing the ANOVA test
AnovaResults = f_oneway(*CategoryGroupLists)
print('P-Value for Anova is: ', AnovaResults[1])
print(AnovaResults)

```

P-Value for Anova is: 4.3558466424602e-12

F_onewayResult(statistic=155.65229110512132, pvalue=4.3558466424602e-12)



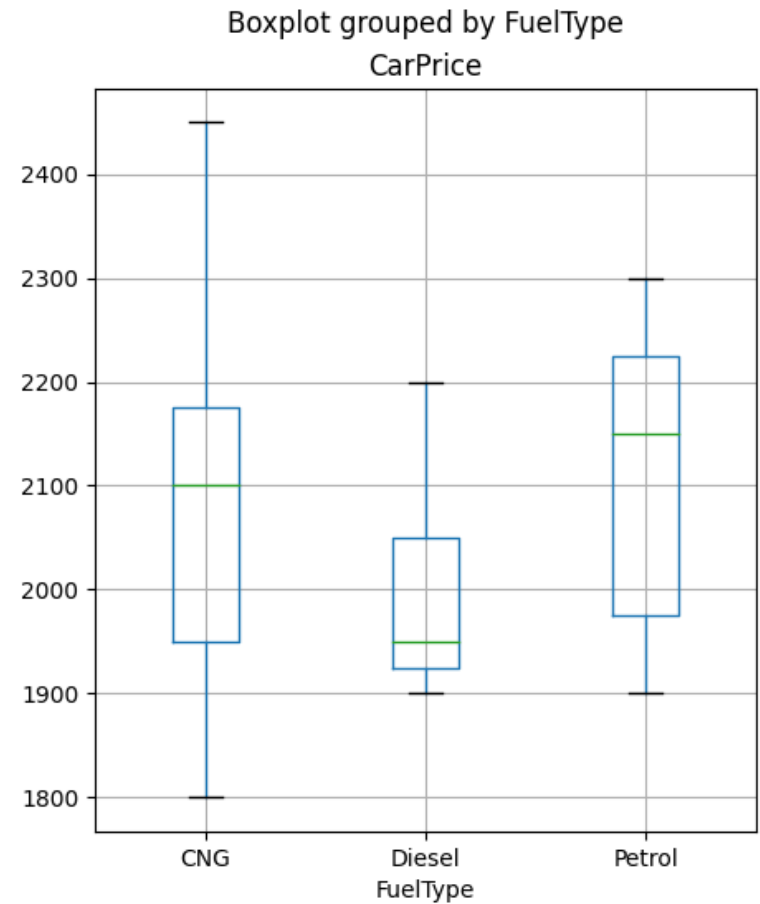
```

# Generating sample data
import pandas as pd
ColumnNames=['FuelType','CarPrice']
DataValues= [[ 'Petrol', 2000],
               [ 'Petrol', 2300],
               [ 'Petrol', 1900],
               [ 'Petrol', 2150],
               [ 'Petrol', 2200],
               [ 'Petrol', 2250],
               [ 'Petrol', 1950],
               [ 'Diesel', 1900],
               [ 'Diesel', 2200],
               [ 'Diesel', 1950],
               [ 'Diesel', 1950],
               [ 'Diesel', 1900],
               [ 'Diesel', 2100],
               [ 'Diesel', 2000],
               [ 'CNG', 2100],
               [ 'CNG', 2000],
               [ 'CNG', 1900],
               [ 'CNG', 2450],
               [ 'CNG', 2150],
               [ 'CNG', 1800],
               [ 'CNG', 2200]]

#Create the Data Frame
CarData=pd.DataFrame(data=DataValues,columns=ColumnNames)
print(CarData)

# Generating boxplot for CarPrice Vs FuelType
CarData.boxplot(column='CarPrice', by='FuelType', figsize=(5,6))

```



```

# f_oneway() function takes the group data as input and
# returns F-statistic and P-value
from scipy.stats import f_oneway

# Running the one-way anova test between CarPrice and
FuelTypes
# Assumption(H0) is that FuelType and CarPrices are NOT
correlated

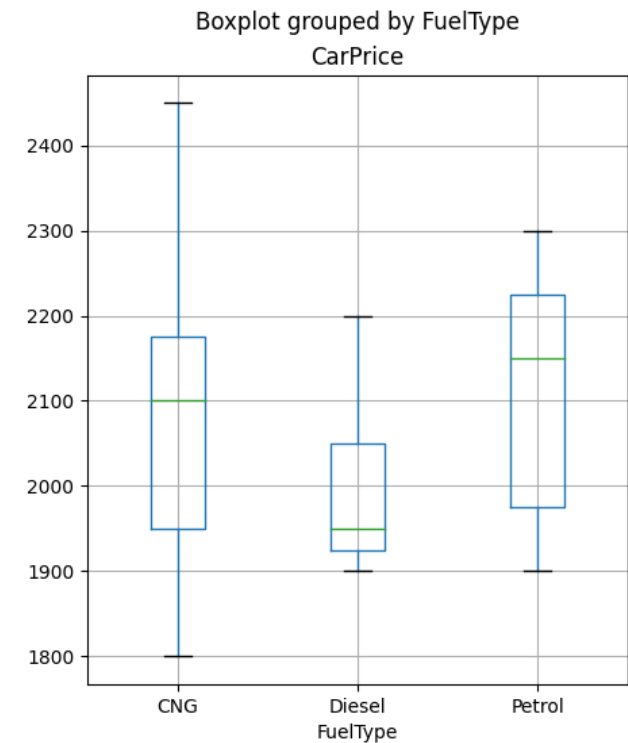
# Finds out the Prices data for each FuelType as a list
CategoryGroupLists=CarData.groupby('FuelType')['CarPrice'].apply(list)

# Performing the ANOVA test
AnovaResults = f_oneway(*CategoryGroupLists)
print('P-Value for Anova is: ', AnovaResults[1])
print(AnovaResults)

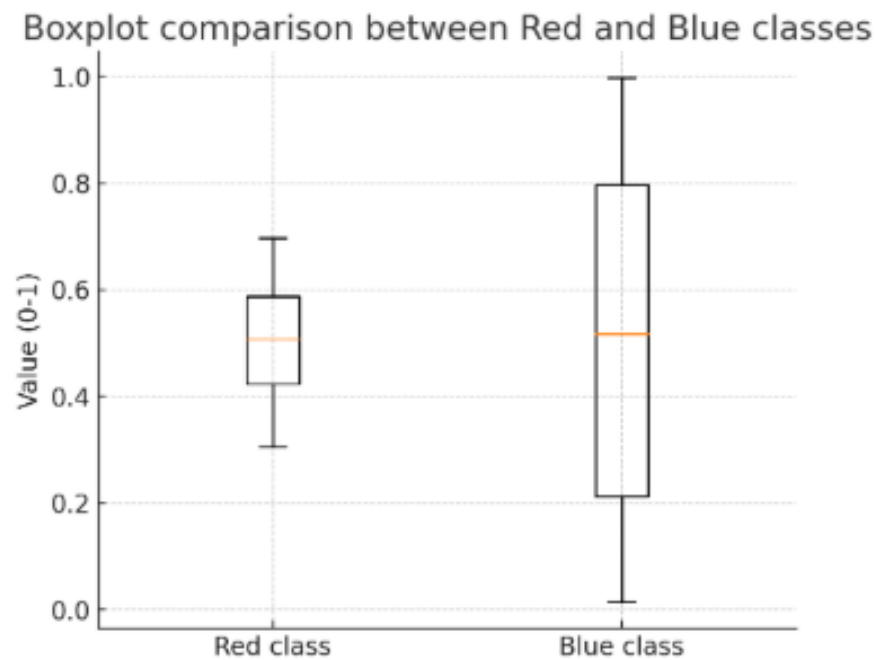
```

P-Value for Anova is: 0.45748021716275267

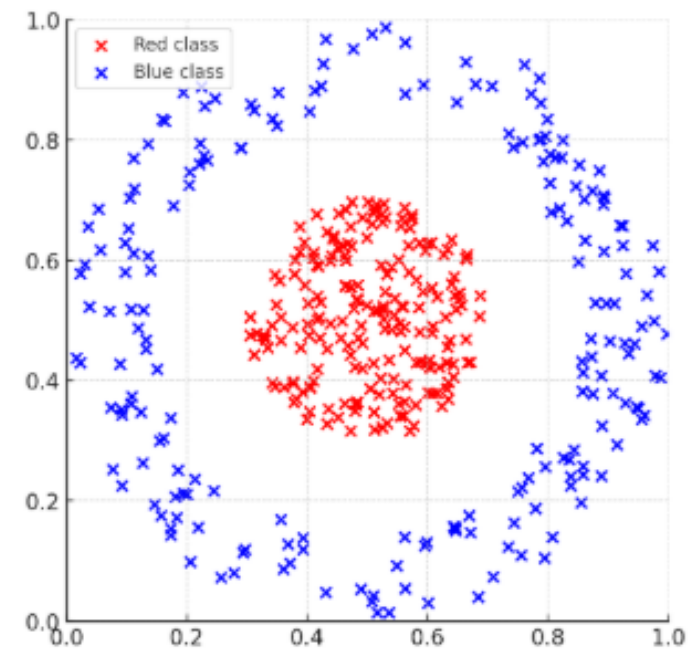
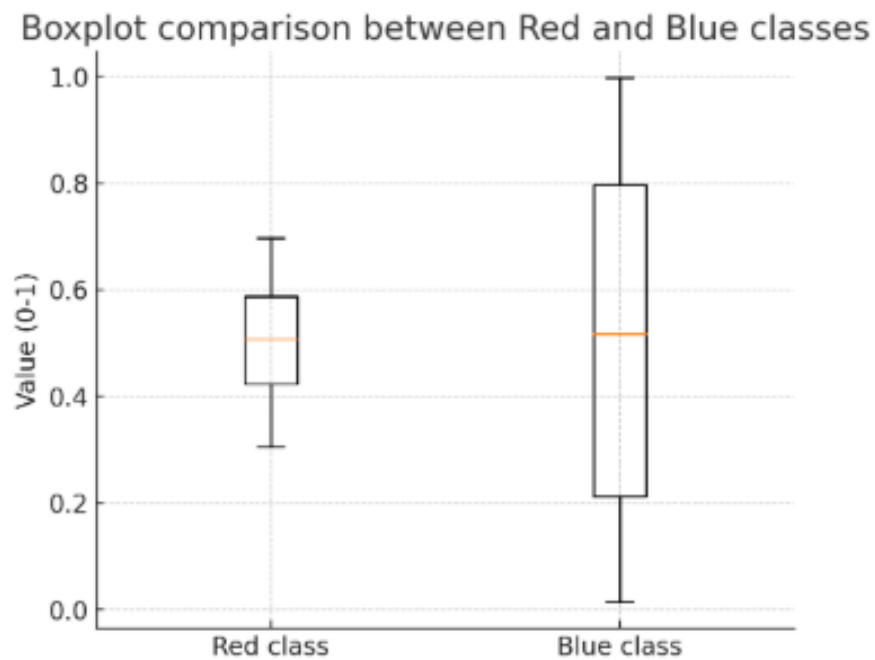
F_onewayResult(statistic=0.8170028818443805, pvalue=0.45748021716275267)



Can it be concluded that there is no relationship?



Can it be concluded that there is no relationship?

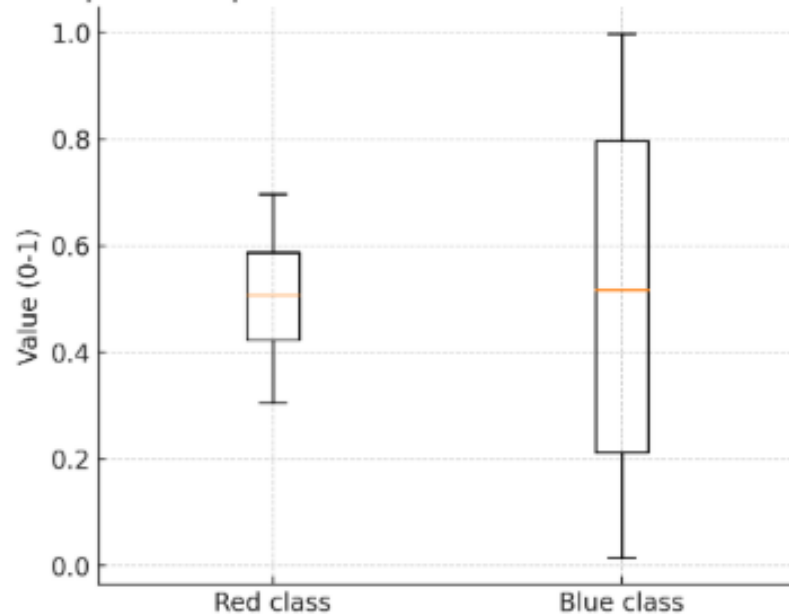


Can it be concluded that there is no relationship?

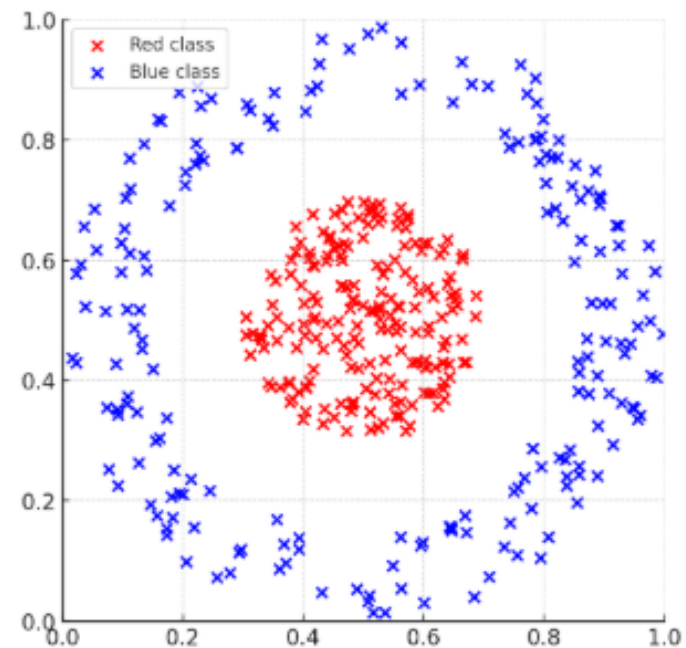
If class balance = 50:50 $\rightarrow I_{\max} = \log 2 = 1 \text{ bit} = 0.693$

If class imbalance (e.g., 70:30) $\rightarrow I_{\max} = H(Y)$

Boxplot comparison between Red and Blue classes



- Feature X: $F = 1.3415$, $p = 0.2475$
- Feature Y: $F = 0.7148$, $p = 0.3984$



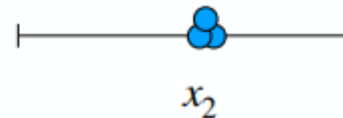
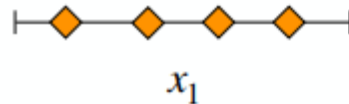
จากค่า Mutual Information (MI) ที่ได้

- Feature X = 0.4566
- Feature Y = 0.3164

Feature with Low Variance



- A simple baseline approach to feature selection.
- It removes all features whose variance doesn't meet some threshold.
- At the very least, we should remove all zero-variance features, i.e. features that have the same value in all samples.



Low Variance Feature Removal



- A simple baseline approach to feature selection.
- It removes all features whose variance doesn't meet some threshold.
- At the very least, we should remove all zero-variance features, i.e. features that have the same value in all samples.
- As an example, suppose that we have a dataset with boolean features, and we want to remove all features that are either one or zero (on or off) in more than 80% of the samples.
- Boolean features are Bernoulli random variables, and the variance of such variables is given by

$$\text{Var}[X] = p(1 - p)$$

Low Variance Feature Removal

In sklearn, you may input the desired level of variance for **VarianceThreshold** selector.

```
In [ ]: from sklearn.feature_selection import VarianceThreshold
sel = VarianceThreshold(threshold=(.8 * (1 - .8)))
sel.fit_transform(X)
```

- sklearn will automatically drop the columns where variance does not meet the desired level.
- Note that this can take care of one-hot encoded features with small number of ones.

VarianceThreshold $0.8 \times (1 - 0.8) = 0.16$

```
from sklearn.preprocessing import OneHotEncoder

X = [['blue'], ['green'], ['blue'], ['blue'], ['green'], ['red'], ['blue'], ['green']]
y = [0, 0, 1, 0, 0, 0, 1, 0]

enc = OneHotEncoder(drop='first')
enc.fit(X)
X_ohc = enc.transform(X)
X_ohc.toarray()
```

```
array([[0., 0.],
       [1., 0.],
       [0., 0.],
       [0., 0.],
       [1., 0.],
       [0., 1.],
       [0., 0.],
       [1., 0.]])
```

```
from sklearn.feature_selection import VarianceThreshold

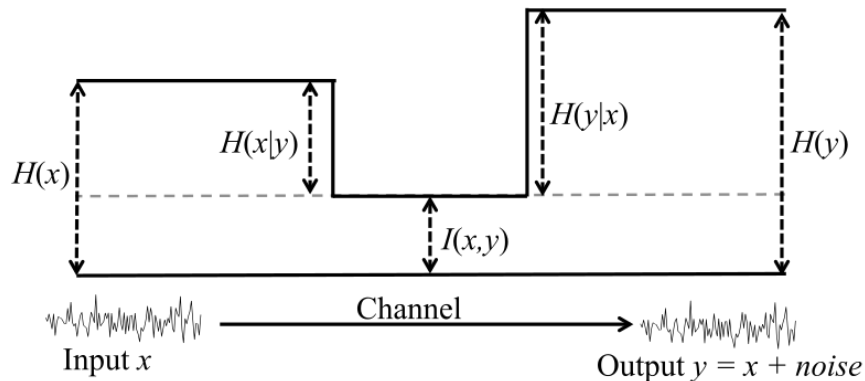
sel = VarianceThreshold(threshold=(.8 * (1 - .8)))

sel.fit(X_ohc)
sel.transform(X_ohc).toarray()
```

```
array([[0.],
       [1.],
       [0.],
       [0.],
       [1.],
       [0.],
       [0.],
       [1.]])
```

Mutual Information

- Mutual Information $I(X;Y)$ is the same as information gain, it is the information variable X and Y share or relevancy between two variables.
- Mutual information is symmetric, $I(X;Y)$ is the same as $I(Y;X)$.



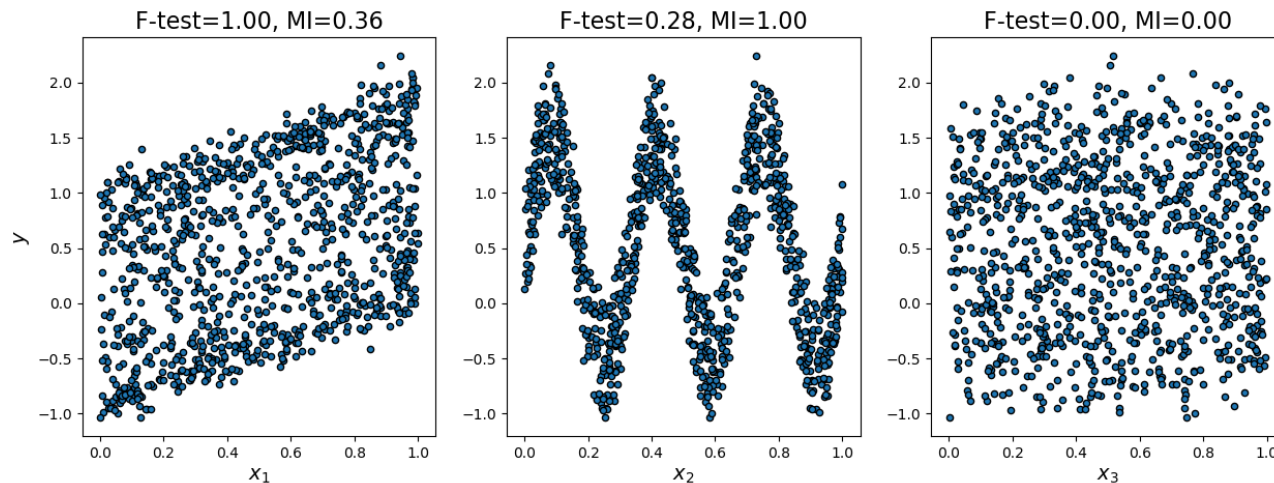
$$\begin{aligned} I(X; Y) &\equiv H(X) - H(X|Y) \\ &\equiv H(Y) - H(Y|X) \\ &\equiv H(X) + H(Y) - H(X, Y) \\ &\equiv H(X, Y) - H(X|Y) - H(Y|X) \end{aligned}$$

$$H(X, Y) = - \sum_x \sum_y P(x, y) \log_2 [P(x, y)]$$

F-Test vs Mutual Information



- As F-test captures only linear dependency, it rates x_1 as the most discriminative feature.
- On the other hand, mutual information can capture any kind of dependency between variables and it rates x_2 as the most discriminative feature, which probably agrees better with our intuitive perception for this example.
- Both methods correctly marks x_3 as irrelevant.



Mutual Information



- Mutual information methods can capture any kind of statistical dependency, but being nonparametric, they require more samples for accurate estimation.
- Mutual information is a non-negative value, the higher the more information shared between two variables.
- Can be calculated for both regression and classification problem.
- sklearn mutual_info_regression: http://scikit-learn.org/stable/modules/generated/sklearn.feature_selection.mutual_info_regression.html
- sklearn mutual_info_classif: http://scikit-learn.org/stable/modules/generated/sklearn.feature_selection.mutual_info_classif.html

Mutual Information



- For mutual information, you might need to identify each feature as discrete or continuous before proceeding.
- Unlike F-Test, mutual information does not have any p-value, so you may not know whether a particular value of MI is significant or not. They are used as feature ranking measure, not thresholded measure.

Chi-squared Test



- For two categorical variables with r classes and c classes respectively, you can make a contingency table that has r rows and c columns.
- The chi square test can be thought of as a test of independence. In a test of independence the null and alternative hypotheses are:
 - H_0 : The two categorical variables are independent.
 - H_a : The two categorical variables are related.

<http://math.hws.edu/javamath/ryan/ChiSquare.html>

Chi-square Test

- The Chi-square test is used to assess the relationship between categorical variables by comparing observed frequencies with expected frequencies.
- **Hypotheses:**
 - H_0 (Null Hypothesis): There is no relationship between the categorical variables.
 - H_1 (Alternative Hypothesis): There is a relationship between the categorical variables.
- **Chi-square statistic (χ^2):** Calculated by summing the squared differences between observed and expected frequencies. A higher chi-square value indicates a stronger relationship between variables.
- **p-value:** Used to assess the likelihood that the observed result occurred by chance. If the p-value is less than 0.05, it indicates a statistically significant relationship between the variables.
- **Usage:** Chi-square is commonly used to test the association between two categorical variables, such as testing whether smoking is related to heart disease.

Chi-squared Test



- Example: we want to know whether continents (X) impact the frequency of different types of malaria (Y).

	Asia	Africa	South America	Total
Malaria A	31	14	45	90
Malaria B	2	5	53	60
Malaria C	53	45	2	100
Totals	86	64	100	250

Chi-squared Test

- We can use the equation to compute χ^2

$$\chi_c^2 = \sum_i \frac{(FO_i - FE_i)^2}{FE_i}$$

	Asia	Africa	South America	Total
Malaria A	31	14	45	90
Malaria B	2	5	53	60
Malaria C	53	45	2	100
Totals	86	64	100	250

(90/250)*(86/250)*250

Observed	Expected	O - E	(O - E) ²	(O - E) ² /E
31	30.96	0.04	0.0016	0.0000516
14	23.04	9.04	81.72	3.546
45	36.00	9.00	81	2.25
2	20.64	18.64	347.45	16.83
5	15.36	10.36	107.33	6.99
53	24.00	29.00	841	35.04
53	34.40	18.60	345.96	10.06
45	25.60	19.40	376.36	14.7
2	40.00	38.00	1444.00	36.1

Chi Square = 125.516

<http://math.hws.edu/javamath/ryan/ChiSquare.html>

```
ColumnNames = ['CIBIL', 'AGE', 'GENDER', 'SALARY', 'APPROVE_LOAN']
```

```
DataValues = [
    [480, 28, 'M', 610000, 'No'],
    [480, 42, 'M', 140000, 'No'],
    [480, 29, 'F', 420000, 'No'],
    [490, 30, 'M', 420000, 'No'],
    [500, 27, 'M', 420000, 'No'],
    [510, 34, 'F', 190000, 'No'],
    [550, 24, 'M', 330000, 'No'],
    [560, 34, 'M', 160000, 'No'],
    [560, 25, 'F', 300000, 'Yes'],
    [570, 34, 'M', 450000, 'No'],
    [590, 30, 'F', 140000, 'Yes'],
    [600, 33, 'M', 600000, 'No'],
    [600, 22, 'M', 400000, 'No'],
    [600, 25, 'F', 490000, 'Yes'],
    [610, 32, 'M', 120000, 'Yes'],
    [630, 29, 'F', 360000, 'Yes'],
    [630, 30, 'M', 480000, 'Yes'],
    [660, 29, 'F', 460000, 'Yes'],
    [700, 32, 'M', 470000, 'No'],
    [740, 28, 'F', 400000, 'Yes']
]
```

```
#Create the Data Frame
```

```
LoanData=pd.DataFrame(data=DataValues,columns=ColumnNames)
```

```
print(LoanData)
```

```
#####
```

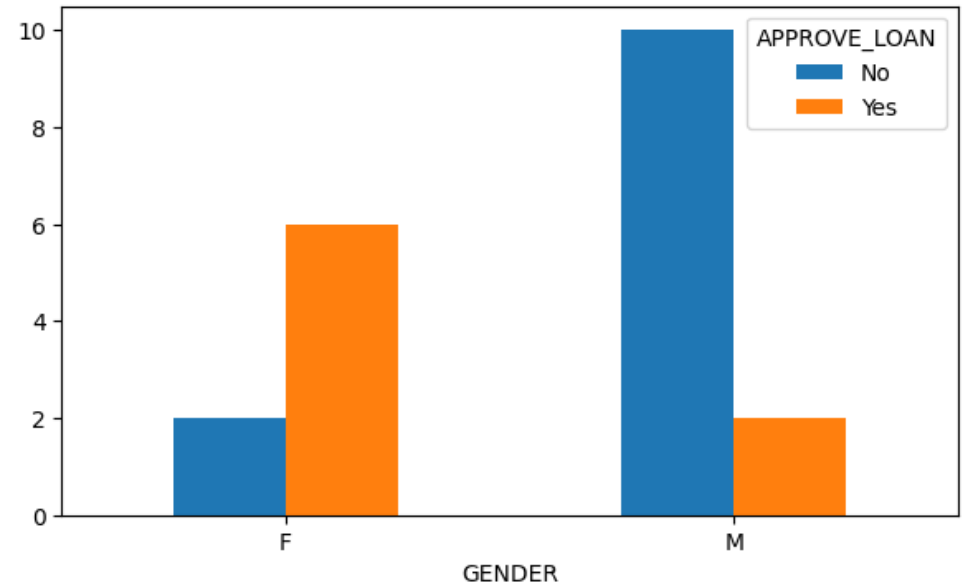
```
# Cross tabulation between GENDER and APPROVE_LOAN
```

```
CrosstabResult=pd.crosstab(index=LoanData['GENDER'],columns=LoanData['APPROVE_LOAN'])
```

```
print(CrosstabResult)
```

```
# Grouped bar chart between GENDER and APPROVE_LOAN
```

```
CrosstabResult.plot.bar(figsize=(7,4), rot=0)
```



```

# Cross tabulation between GENDER and APPROVE_LOAN
CrosstabResult=pd.crosstab(index=LoanData['GENDER'],columns=LoanData['APPROVE_LOAN'])
print(CrosstabResult)

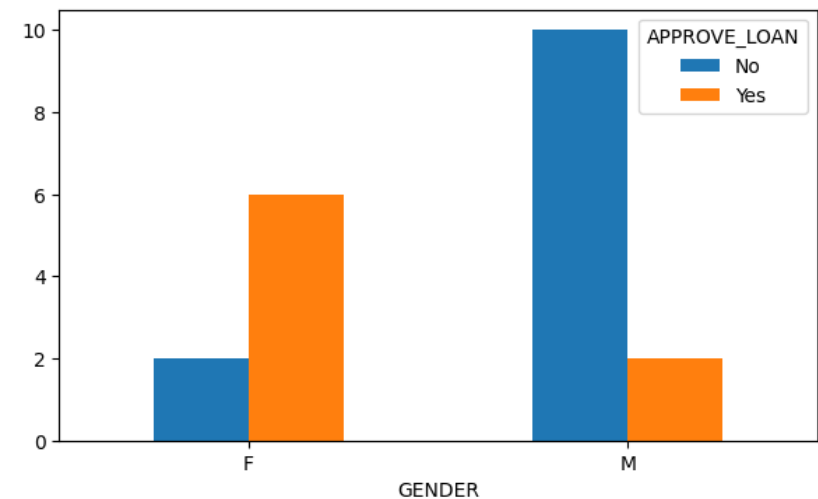
# importing the required function
from scipy.stats import chi2_contingency

# Performing Chi-sq test
ChiSqResult = chi2_contingency(CrosstabResult)

# P-Value is the Probability of H0 being True

print('The P-Value of the ChiSq Test is:', ChiSqResult[1])
print(ChiSqResult)

```



P-Value of the ChiSq Test is: 0.03212125119157755
 Chi2ContingencyResult(statistic=4.592013888888888, pvalue=0.03212125119157755,
 dof=1, expected_freq=array([[4.8, 3.2], [7.2, 4.8]]))

```

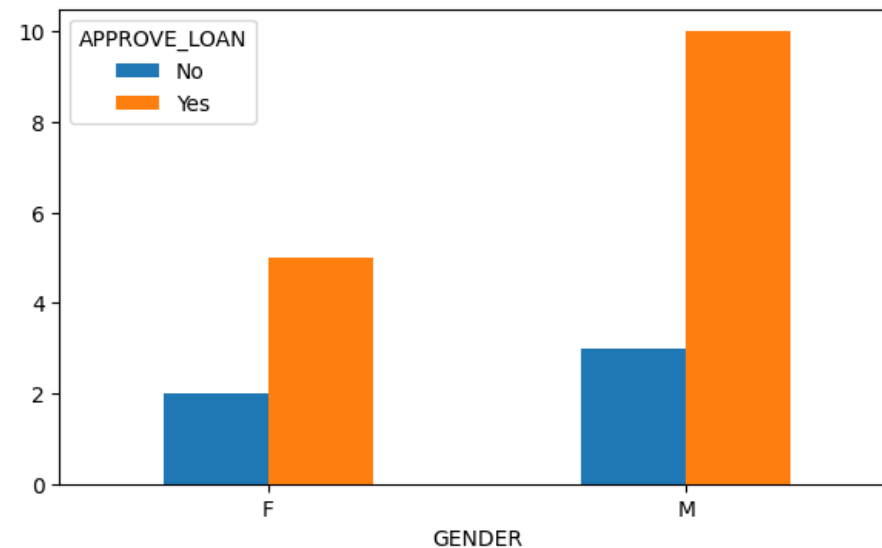
# Creating a sample data frame
import pandas as pd
ColumnNames=[ 'CIBIL', 'AGE', 'GENDER', 'SALARY', 'APPROVE_LOAN']
DataValues=[ [480, 28, 'M', 610000, 'Yes'],
[480, 42, 'M', 140000, 'No'],
[480, 29, 'F', 420000, 'No'],
[490, 30, 'M', 420000, 'No'],
[500, 27, 'M', 420000, 'No'],
[510, 34, 'F', 190000, 'No'],
[550, 24, 'M', 330000, 'Yes'],
[560, 34, 'M', 160000, 'Yes'],
[560, 25, 'F', 300000, 'Yes'],
[570, 34, 'M', 450000, 'Yes'],
[590, 30, 'F', 140000, 'Yes'],
[600, 33, 'M', 600000, 'Yes'],
[600, 22, 'M', 400000, 'Yes'],
[600, 25, 'F', 490000, 'Yes'],
[610, 32, 'M', 120000, 'Yes'],
[630, 29, 'F', 360000, 'Yes'],
[630, 30, 'M', 480000, 'Yes'],
[660, 29, 'F', 460000, 'Yes'],
[700, 32, 'M', 470000, 'Yes'],
[740, 28, 'M', 400000, 'Yes']]

#Create the Data Frame
LoanData=pd.DataFrame(data=DataValues,columns=ColumnNames)
print(LoanData)

#####
# Cross tabulation between GENDER and APPROVE_LOAN
CrosstabResult=pd.crosstab(index=LoanData['GENDER'],columns=LoanData['APPROVE_LOAN'])
print(CrosstabResult)

# Grouped bar chart between GENDER and APPROVE_LOAN
CrosstabResult.plot.bar(figsize=(7,4), rot=0)

```



```

# Cross tabulation between GENDER and APPROVE_LOAN
CrosstabResult=pd.crosstab(index=LoanData['GENDER'],columns=LoanData['APPROVE_LOAN'])
print(CrosstabResult)

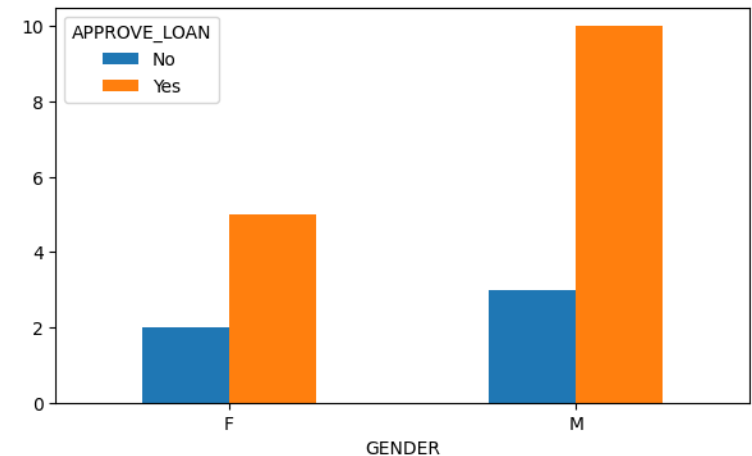
# importing the required function
from scipy.stats import chi2_contingency

# Performing Chi-sq test
ChiSqResult = chi2_contingency(CrosstabResult)

# P-Value is the Probability of H0 being True

print('The P-Value of the ChiSq Test is:', ChiSqResult[1])
print(ChiSqResult)

```



The P-Value of the ChiSq Test is: 1.0
 Chi2ContingencyResult(statistic=0.0, pvalue=1.0, dof=1,
 expected_freq=array([[1.75, 5.25], [3.25, 9.75]]))

Chi-squared Test



- Similar to F-Test, Chi-Squares measure must be compared against a lookup table to determine p-value (statistical significance of the chi-square numbers).
- You can then use this p-value to assess importance of your features.
- Note that Chi-Squared works best when both features and targets are categorical. There is no continuous version. Features must be non-negatives, such as booleans or frequencies.
- more info: http://scikit-learn.org/stable/modules/generated/sklearn.feature_selection.chi2.html

Wrapper Method



- Recursive Feature Elimination is an example of wrapper methods for feature selection.
- Given an external estimator that assigns weights to features (e.g., the coefficients of a linear model)
- Recursive feature elimination (RFE) is to select features by recursively considering smaller and smaller sets of features.
- First, the estimator is trained on the initial set of features and the importance of each feature is obtained either through a `coef_` attribute or through a `feature_importances_` attribute.
- Then, the least important features are pruned from current set of features. That procedure is recursively repeated on the pruned set until the desired number of features to select is eventually reached.

Wrapper Method



```
from sklearn.feature_selection import RFE
from sklearn import linear_model
from sklearn.datasets import load_digits

digits = load_digits()
X = digits.images.reshape((len(digits.images), -1))
y = digits.target

est = linear_model.LinearRegression()
rfe = RFE(estimator=est, n_features_to_select=1, step=1)
rfe.fit(X, y)
ranking = rfe.ranking_.reshape(digits.images[0].shape)
```

RFE Code example (1)

Dataset Preparation

```
[3]: import pandas as pd
import numpy as np

df_wine = pd.read_csv('https://archive.ics.uci.edu/
    'ml/machine-learning-databases/wine/wine.data',
    header=None)

df_wine.columns = ['Class label', 'Alcohol', 'Malic acid', 'Ash',
    'Alcalinity of ash', 'Magnesium', 'Total phenols',
    'Flavanoids', 'Nonflavanoid phenols', 'Proanthocyanins',
    'Color intensity', 'Hue', 'OD280/OD315 of diluted wines',
    'Proline']

print('Class labels', np.unique(df_wine['Class label']))
df_wine.head()
```

Class labels [1 2 3]

```
[3]:
```

	Class label	Alcohol	Malic acid	Ash	Alcalinity of ash	Magnesium	Total phenols	Flavanoids	Nonflavanoid phenols	Proanthocyanins	Color intensity	Hue	OD280/OD315 of diluted wines	Proline
0	1	14.23	1.71	2.43	15.6	127	2.80	3.06	0.28	2.29	5.64	1.04	3.92	1065
1	1	13.20	1.78	2.14	11.2	100	2.65	2.76	0.26	1.28	4.38	1.05	3.40	1050
2	1	13.16	2.36	2.67	18.6	101	2.80	3.24	0.30	2.81	5.68	1.03	3.17	1185
3	1	14.37	1.95	2.50	16.8	113	3.85	3.49	0.24	2.18	7.80	0.86	3.45	1480
4	1	13.24	2.59	2.87	21.0	118	2.80	2.69	0.39	1.82	4.32	1.04	2.93	735

RFE Code example (2)

```
[4]: from sklearn.model_selection import train_test_split

X, y = df_wine.iloc[:, 1:].values, df_wine.iloc[:, 0].values

X_train, X_test, y_train, y_test = \
    train_test_split(X, y,
                    test_size=0.3,
                    random_state=0,
                    stratify=y)
```

```
[5]: from sklearn.preprocessing import StandardScaler

sc = StandardScaler()
X_train_std = sc.fit_transform(X_train)
X_test_std = sc.transform(X_test)
```

RFE Code example (3)

Recursive Feature Selection

```
[6]: from sklearn.linear_model import LogisticRegression
      from sklearn.feature_selection import RFE

      lr = LogisticRegression(solver='liblinear', random_state=123)

      rfe = RFE(estimator=lr, n_features_to_select=5, step=1)
      rfe.fit(X_train_std, y_train)

      X_train_sub = rfe.transform(X_train_std)
```

Which features are selected?

```
[7]: rfe.support_

[7]: array([ True, False, False, False, False, False,  True, False, False,
          True,  True, False,  True])

[8]: df_wine.columns[1:][rfe.support_]

[8]: Index(['Alcohol', 'Flavanoids', 'Color intensity', 'Hue', 'Proline'], dtype='object')
```

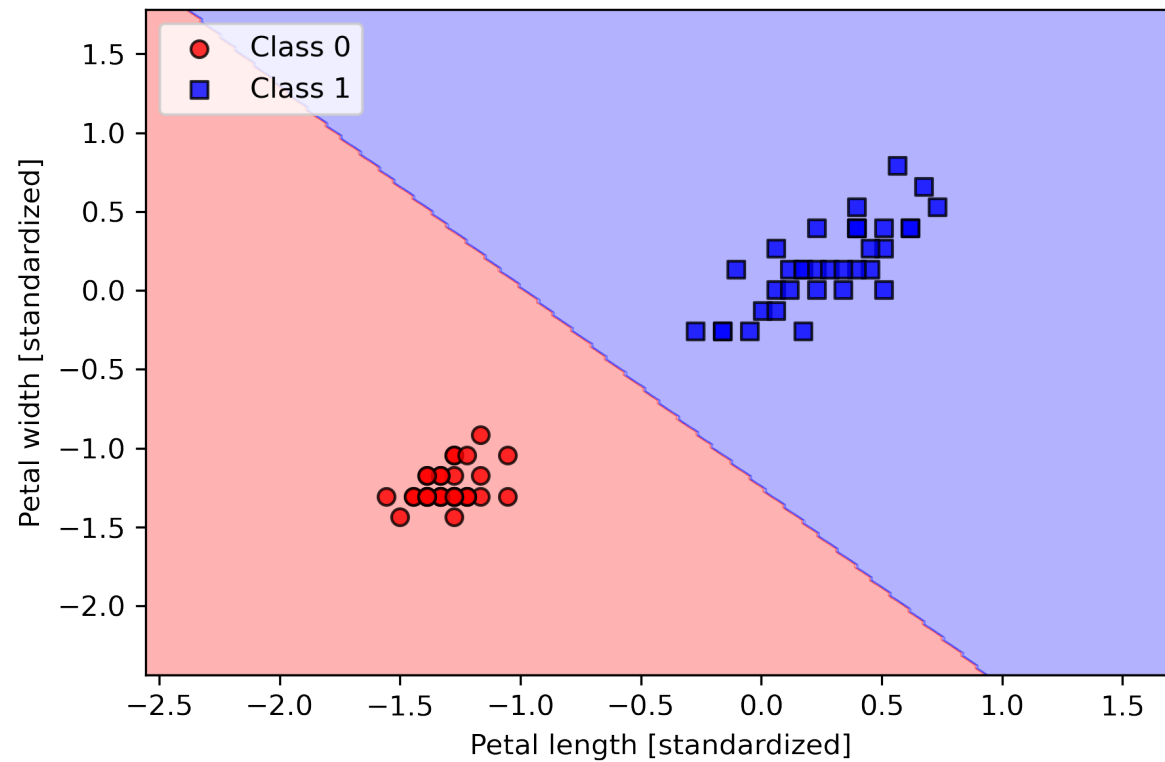
Wrapper Method



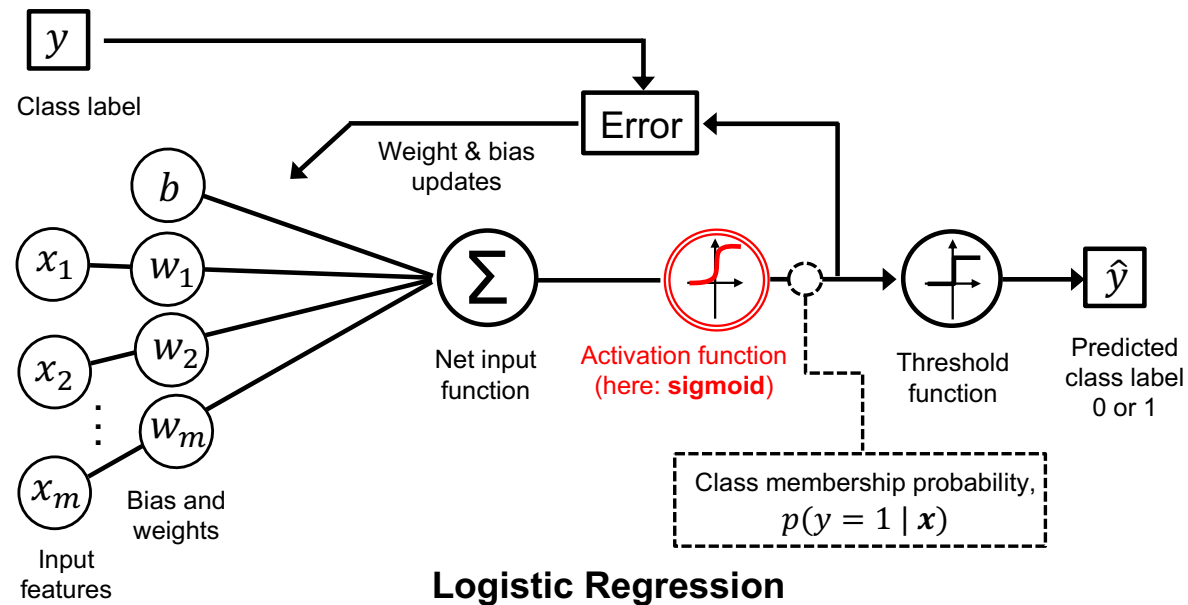
- It is important to consider feature selection a part of the model selection process. If you do not, it may lead to overfitting.
- Include feature selection within the inner-loop when you do cross-validation.
- A mistake would be to perform feature selection first to prepare your data, then perform model selection and training on the selected features.
- If you perform feature selection on all of the data and then cross-validate, then the test data in each fold of the cross-validation procedure was also used to choose the features and this is what biases the performance analysis.

Embedded Method

Logistic Regression



Logistic Regression



Source: Raschka, Liu, and Mirjalili. *Machine Learning with PyTorch and Scikit-Learn*, Ch 3

Logistic Regression Hyperparameters

Regular loss function to minimize during training

$$L(\mathbf{w}, b \mid \mathbf{x}) = - \sum_{i=1} \left[y^{(i)} \log \left(\sigma \left(z^{(i)} \right) \right) + (1 - y^{(i)}) \log \left(1 - \sigma \left(z^{(i)} \right) \right) \right]$$

L1 Norm /

LASSO (Least Absolute Shrinkage and Selection Operator)

$$\text{L1 norm: } \lambda ||\mathbf{w}||_1 = \lambda \sum_{j=1}^m |w_j|$$

L1 Regularization / LASSO (Embedded)

Least Absolute Shrinkage and Selection Operator

L1 penalty against complexity

$$\lambda ||\mathbf{w}||_1 = \lambda \sum_{j=1}^m |w_j|$$

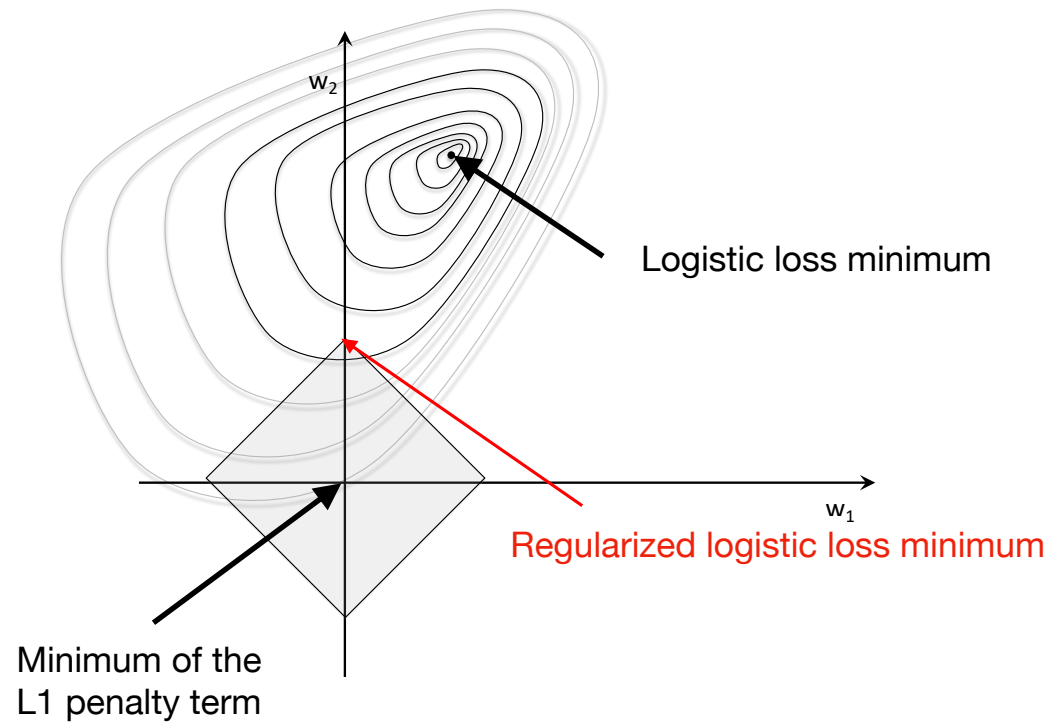
hyperparameter

L1-penalized loss

$$L_{L1}(\mathbf{w}, b \mid \mathbf{x}) = - \sum_{i=1} \left[y^{(i)} \log \left(\sigma \left(z^{(i)} \right) \right) + (1 - y^{(i)}) \log \left(1 - \sigma \left(z^{(i)} \right) \right) \right] + \lambda ||\mathbf{w}||_1$$

L1 Regularization / LASSO (Embedded)

Least Absolute Shrinkage and Selection Operator



For more details, see Tibshirani, Ryan, and L. Wasserman. "A closer look at sparse regression." *Lecture notes* (2016).

L1 Regularization / LASSO (Embedded)

Least Absolute Shrinkage and Selection Operator

Wine Dataset

<https://archive.ics.uci.edu/ml/machine-learning-databases/wine/wine.data>

'Class lab	'Alcohc	'Malic aci	'Ash	Alcalini of ash'	'Magnesi	'Total phen	'Flavanoid	'Nonflavan phenols'	'Proanthocyan	'Color intens	'Hue OD280/OD31 of diluted win	'Proline																					
1,14.23,1.71,2.43,15.6,127,2.8,3.06,.28,2.29,5.64,1.04,3.92,1065	1,13.2,1.78,2.14,11.2,100,2.65,2.76,.26,1.28,4.38,1.05,3.4,1050	1,13.16,2.36,2.67,18.6,101,2.8,3.24,.3,2.81,5.68,1.03,3.17,1185	1,14.37,1.95,2.5,16.8,113,3.85,3.49,.24,2.18,7.8,.86,3.45,1480	1,13.24,2.59,2.87,21,118,2.8,2.69,.39,1.82,4.32,1.04,2.93,735	1,14.2,1.76,2.45,15.2,112,3.27,3.39,.34,1.97,6.75,1.05,2.85,1450	1,14.39,1.87,2.45,14.6,96,2.5,2.52,.3,1.98,5.25,1.02,3.58,1290	1,14.06,2.15,2.61,17.6,121,2.6,2.51,.31,1.25,5.05,1.06,3.58,1295	1,14.83,1.64,2.17,14,97,2.8,2.98,.29,1.98,5.2,1.08,2.85,1045	1,13.86,1.35,2.27,16,98,2.98,3.15,.22,1.85,7.22,1.01,3.55,1045	1,14.1,2.16,2.3,18,105,2.95,3.32,.22,2.38,5.75,1.25,3.17,1510	1,14.12,1.48,2.32,16.8,95,2.2,2.43,.26,1.57,5,1.17,2.82,1280	1,13.75,1.73,2.41,16,89,2.6,2.76,.29,1.81,5.6,1.15,2.9,1320	1,14.75,1.73,2.39,11.4,91,3.1,3.69,.43,2.81,5.4,1.25,2.73,1150	1,14.38,1.87,2.38,12,102,3.3,3.64,.29,2.96,7.5,1.2,3,1547	1,13.63,1.81,2.7,17.2,112,2.85,2.91,.3,1.46,7.3,1.28,2.88,1310	1,14.3,1.92,2.72,20,120,2.8,3.14,.33,1.97,6.2,1.07,2.65,1280	1,13.83,1.57,2.62,20,115,2.95,3.4,.4,1.72,6.6,1.13,2.57,1130	1,14.19,1.59,2.48,16.5,108,3.3,3.93,.32,1.86,8.7,1.23,2.82,1680	1,13.64,3.1,2.56,15.2,116,2.7,3.03,.17,1.66,5.1,.96,3.36,845	1,14.06,1.63,2.28,16,126,3,3.17,.24,2.1,5.65,1.09,3.71,780	1,12.93,3.8,2.65,18.6,102,2.41,2.41,.25,1.98,4.5,1.03,3.52,770	1,13.71,1.86,2.36,16.6,101,2.61,2.88,.27,1.69,3.8,1.11,4,1035	1,12.85,1.6,2.52,17.8,95,2.48,2.37,.26,1.46,3.93,1.09,3.63,1015	1,13.5,1.81,2.61,20,96,2.53,2.61,.28,1.66,3.52,1.12,3.82,845	1,13.05,2.05,3.22,25,124,2.63,2.68,.47,1.92,3.58,1.13,3.2,830	1,13.39,1.77,2.62,16.1,93,2.85,2.94,.34,1.45,4.8,.92,3.22,1195	1,13.3,1.72,2.14,17,94,2.4,2.19,.27,1.35,3.95,1.02,2.77,1285	1,13.87,1.9,2.8,19.4,107,2.95,2.97,.37,1.76,4.5,1.25,3.4,915	1,14.02,1.68,2.21,16,96,2.65,2.33,.26,1.98,4.7,1.04,3.59,1035	1,13.73,1.5,2.7,22.5,101,3,3.25,.29,2.38,5.7,1.19,2.71,1285	1,13.58,1.66,2.36,19.1,106,2.86,3.19,.22,1.95,6.9,1.09,2.88,1515	1,13.68,1.83,2.36,17.2,104,2.42,2.69,.42,1.97,3.84,1.23,2.87,990	1,13.76,1.53,2.7,19.5,132,2.95,2.74,.5,1.35,5.4,1.25,3,1235

Figure Selection

```
from sklearn.model_selection import train_test_split

X, y = df_wine.iloc[:, 1:].values, df_wine.iloc[:, 0].values

X_train, X_test, y_train, y_test = \
    train_test_split(X, y,
                    test_size=0.3,
                    random_state=0,
                    stratify=y)
```

```
from sklearn.preprocessing import StandardScaler

sc = StandardScaler()
X_train_std = sc.fit_transform(X_train)
X_test_std = sc.transform(X_test)
```

```

from sklearn.linear_model import LogisticRegression

lr = LogisticRegression(penalty='l1', C=1.0, solver='liblinear', multi_class='ovr')
# Note that C=1.0 is the default. You can increase
# or decrease it to make the regularization effect
# stronger or weaker, respectively.
lr.fit(X_train_std, y_train)
print('Training accuracy:', lr.score(X_train_std, y_train))
print('Test accuracy:', lr.score(X_test_std, y_test))

```

Training accuracy: 1.0
Test accuracy: 1.0

```
lr.intercept_
```

```
array([-1.26363107, -1.21610924, -2.37035486])
```

```
np.set_printoptions(8)
```

```
lr.coef_[lr.coef_!=0].shape
```

```
(23,)
```

```
lr.coef_
```

```

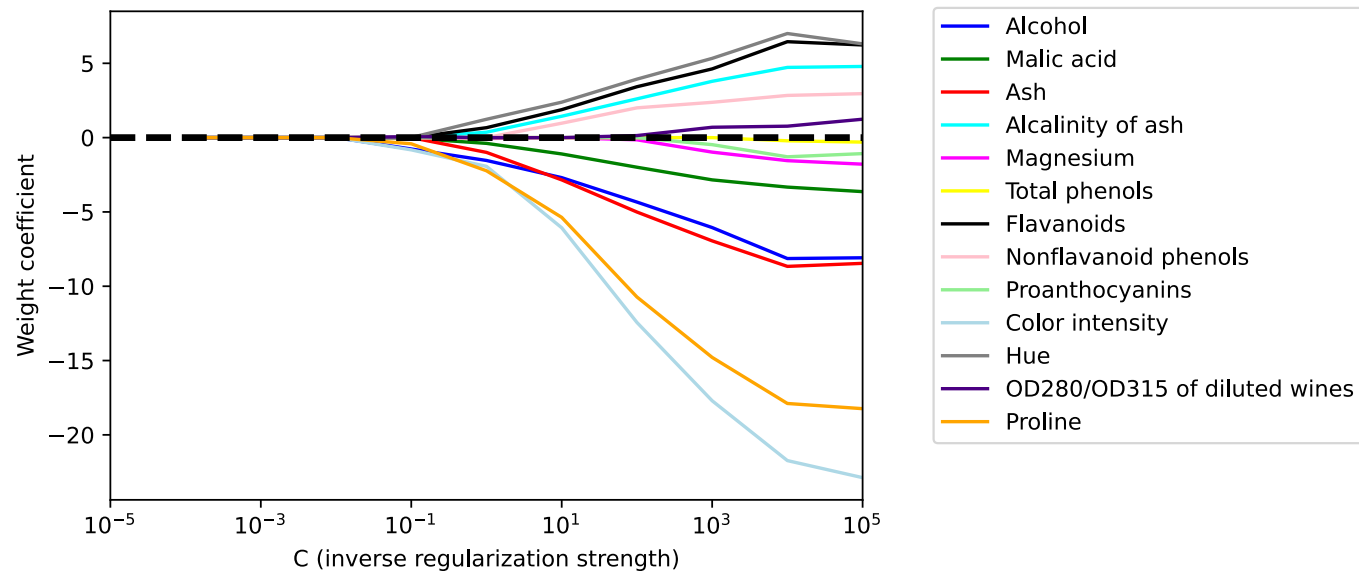
array([[ 1.2455466 ,  0.18072432,  0.74213192, -1.15948629,  0.          ,
         0.          ,  1.17434899,  0.          ,  0.          ,  0.          ,
         0.          ,  0.54353185,  2.51127873],
       [-1.53786975, -0.38667962, -0.9954337 ,  0.36456123, -0.05923747,
         0.          ,  0.66763266,  0.          ,  0.          , -1.93321837,
         1.23529768,  0.          , -2.23229101],
       [ 0.13570425,  0.16821283,  0.35724291,  0.          ,  0.          ,
         0.          , -2.43809231,  0.          ,  0.          ,  1.56377541,
        -0.81940109, -0.49234846,  0.          ]])

```

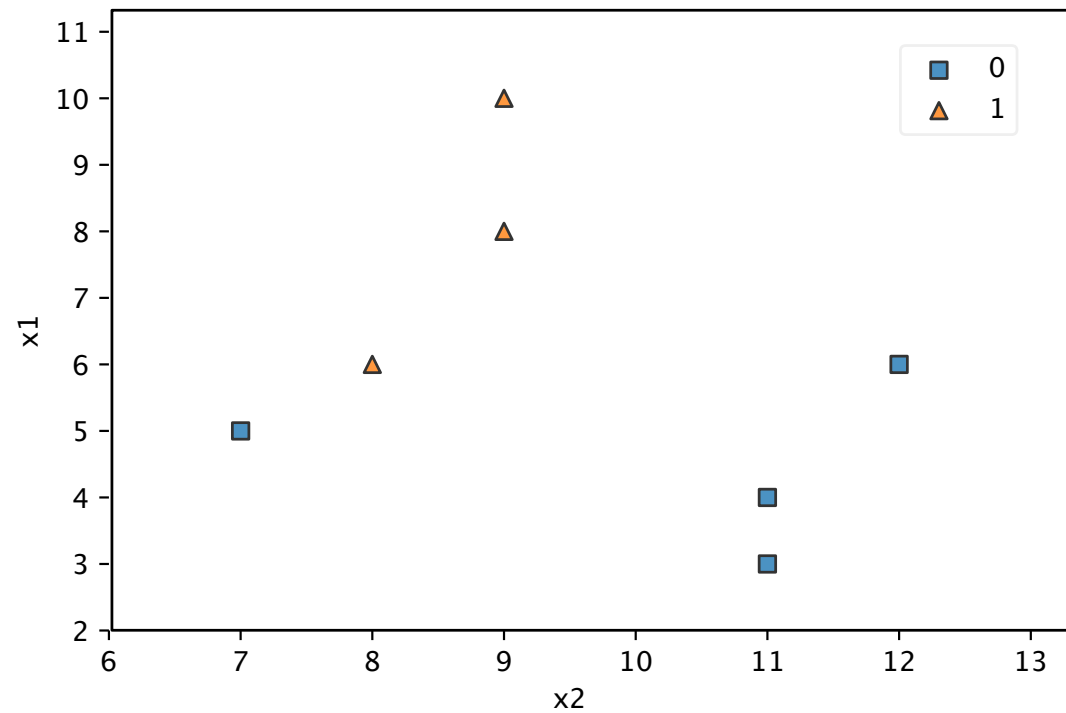
L1 Regularization / LASSO (Embedded)

Least Absolute Shrinkage and Selection Operator

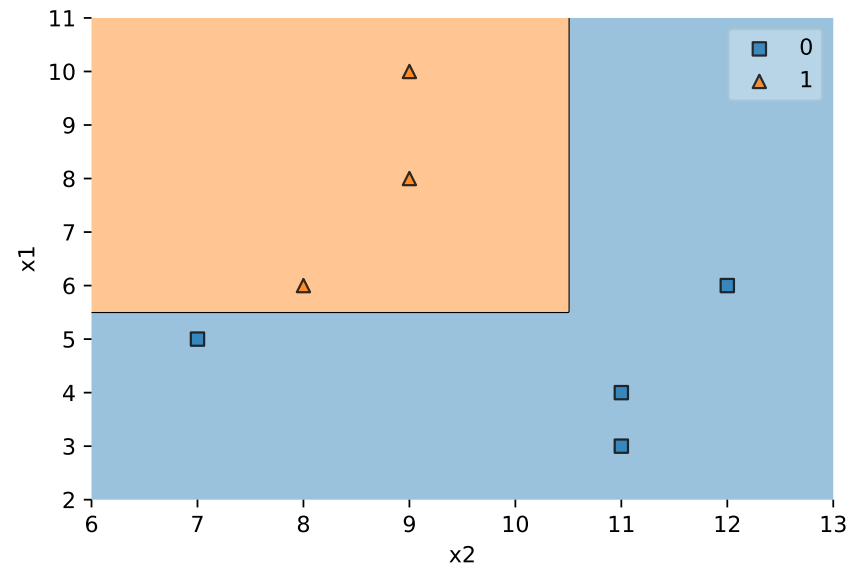
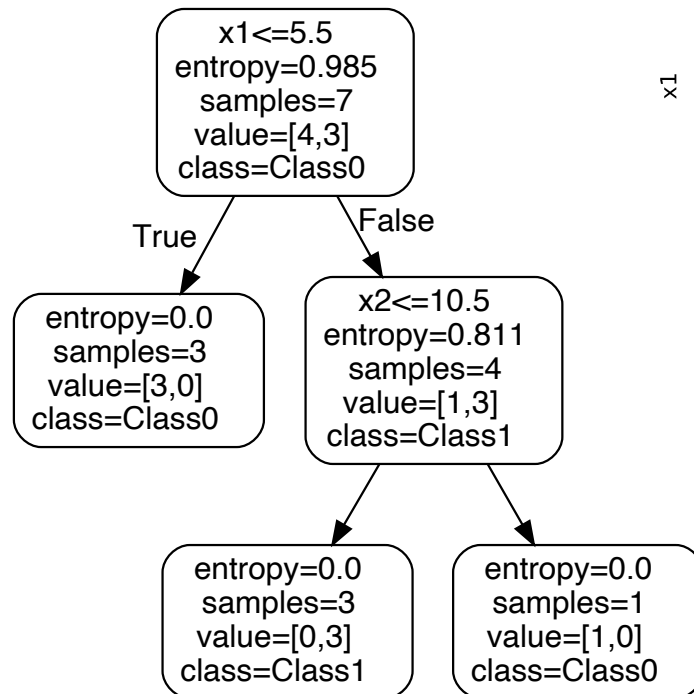
LASSO Path



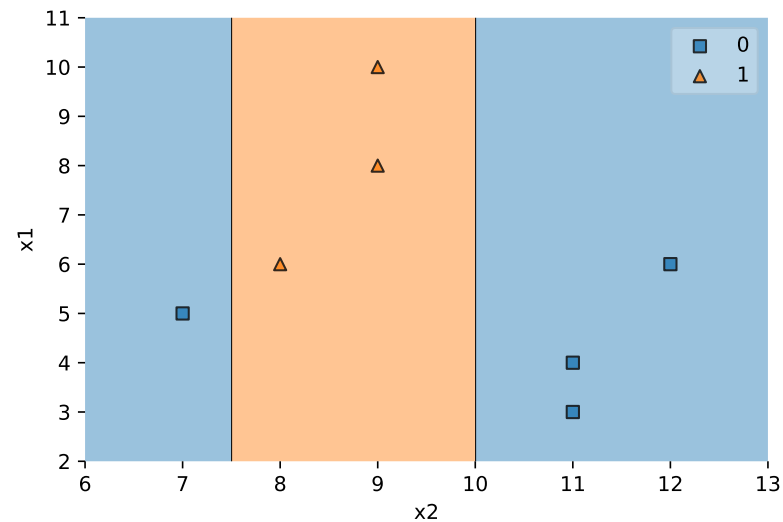
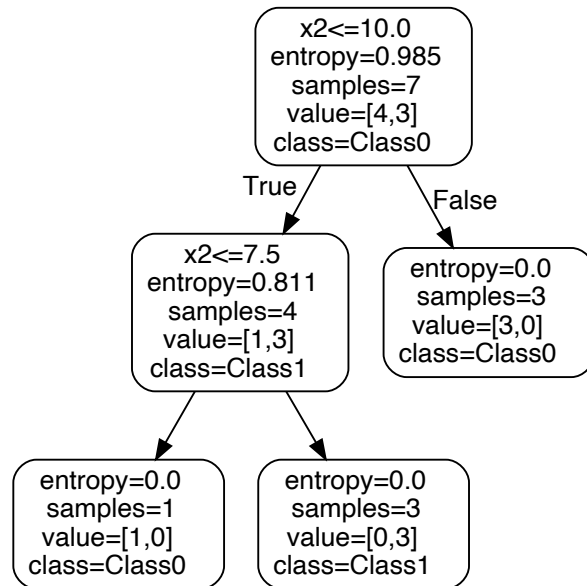
Feature Selection in Decision Trees



Feature Selection in Decision Trees



Feature Selection in Decision Trees



```

import pandas as pd
import numpy as np

df_wine = pd.read_csv('https://archive.ics.uci.edu/
                      'ml/machine-learning-databases/wine/wine.data',
                      header=None)

df_wine.columns = ['Class label', 'Alcohol', 'Malic acid', 'Ash',
                  'Alcalinity of ash', 'Magnesium', 'Total phenols',
                  'Flavanoids', 'Nonflavanoid phenols', 'Proanthocyanins',
                  'Color intensity', 'Hue', 'OD280/OD315 of diluted wines',
                  'Proline']

print('Class labels', np.unique(df_wine['Class label']))
df_wine.head()

```

Class labels [1 2 3]

	Class label	Alcohol	Malic acid	Ash	Alcalinity of ash	Magnesium	Total phenols	Flavanoids	Nonflavanoid phenols	Proanthocyanins	Color intensity	Hue	OD280/OD315 of diluted wines	Proline
0	1	14.23	1.71	2.43	15.6	127	2.80	3.06	0.28	2.29	5.64	1.04	3.92	1065
1	1	13.20	1.78	2.14	11.2	100	2.65	2.76	0.26	1.28	4.38	1.05	3.40	1050
2	1	13.16	2.36	2.67	18.6	101	2.80	3.24	0.30	2.81	5.68	1.03	3.17	1185
3	1	14.37	1.95	2.50	16.8	113	3.85	3.49	0.24	2.18	7.80	0.86	3.45	1480
4	1	13.24	2.59	2.87	21.0	118	2.80	2.69	0.39	1.82	4.32	1.04	2.93	735

```

from sklearn.ensemble import RandomForestClassifier

forest = RandomForestClassifier(n_estimators=500,
                               random_state=1)

forest.fit(X_train, y_train)
importances = forest.feature_importances_

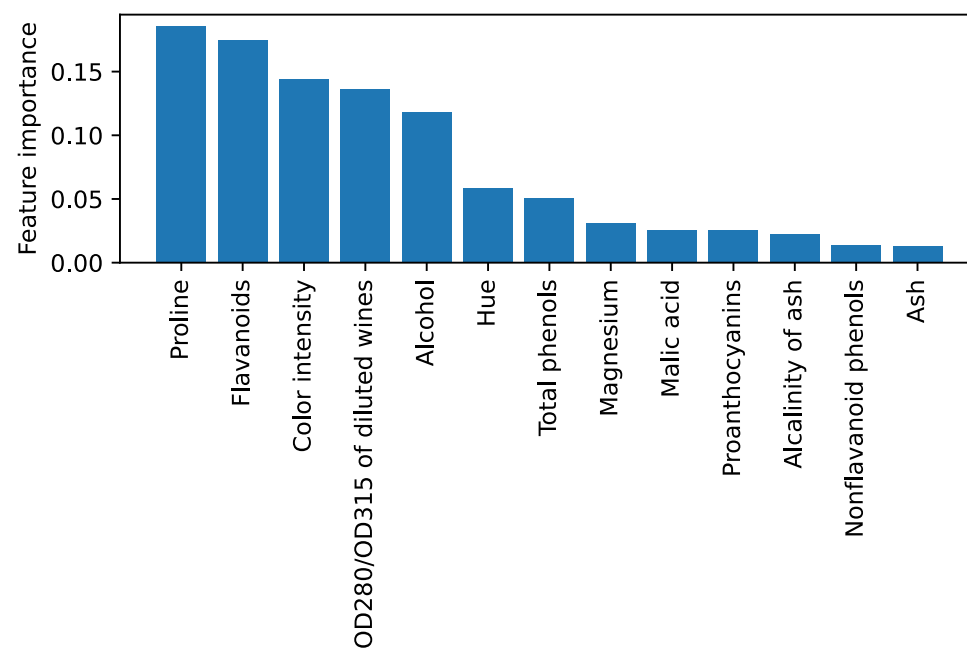
indices = np.argsort(importances)[::-1]

plt.ylabel('Feature importance')
plt.bar(range(X_train.shape[1]),
        importances[indices],
        align='center')

feat_labels = df_wine.columns[1:]
plt.xticks(range(X_train.shape[1]),
           feat_labels[indices], rotation=90)

plt.xlim([-1, X_train.shape[1]])

```



Demo

- Basic Data Preprocessing

- Code: https://drive.google.com/file/d/1Q7CLQ-qltRsGTzhX7W_zuHdyGw9yyczx/view?usp=sharing
- Data: https://drive.google.com/file/d/1Wppkx89FOXqOGUsQL_A4lQx1vKzLeER5/view?usp=drive_link

- PCA

- Code: https://drive.google.com/file/d/1Xk86vKyF8fEMP-a_VtA2QCq9-aP8GewU/view?usp=sharing
 - Data:
 - ConcreteStrength: <https://drive.google.com/file/d/1LIRJ66cSlyqgb0OpIIFvWMCQ5JSm9Prx/view?usp=sharing>
-